



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Βάσεις Δεδομένων

Εξαμηνιαία Εργασία 2025-2026

Σύστημα Διαχείρισης Γενικού Νοσοκομείου «Υγειόπολις»

Μάρκος Καλαβάς

ΑΜ: 03122650

Ανδρέας Κλεάνθους

ΑΜ: 03122661

Μάθημα: Βάσεις Δεδομένων

Καθηγητές: Δημήτρης Τσουμάκος, Μάριος Κόνιαρης

Μάιος 2026

Πίνακας Περιεχομένων

I.	Μέρος A: Σχεδιασμός και Υλοποίηση Βάσης Δεδομένων	2
I.α	ER Διάγραμμα	2
I.β	Σχεσιακό Διάγραμμα και Ανάπτυξη Βάσης Δεδομένων	3
II.	Μέρος B: Ερωτήματα SQL	26
II.α	Ερώτημα 1: Συνολικά έσοδα ανά τμήμα και ανά έτος	26
II.β	Ερώτημα 2: Ιατροί ειδικότητας – εφημερίες και επεμβάσεις	28
II.γ	Ερώτημα 3: Ασθενείς με άνω των 3 νοσηλειών στο ίδιο τμήμα	30
II.δ	Ερώτημα 4: Μέσος όρος αξιολογήσεων ιατρού – EXPLAIN ANALYZE	32
II.ε	Ερώτημα 5: Νέοι ιατροί (<35 ετών) με τις περισσότερες χειρουργικές επεμβάσεις	36
II.ζ	Ερώτημα 6: Ιστορικό νοσηλειών ασθενή – EXPLAIN ANALYZE	38
II.η	Ερώτημα 7: Δραστικές ουσίες – αλλεργίες ασθενών και φάρμακα	45
II.θ	Ερώτημα 8: Προσωπικό χωρίς εφημερία σε συγκεκριμένη ημερομηνία και τμήμα	46
II.ι	Ερώτημα 9: Ασθενείς με ίδιο αριθμό νοσηλευτικών ημερών (>15) εντός έτους	48
II.κ	Ερώτημα 10: Top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα	50
II.λ	Ερώτημα 11: Ιατροί με τουλάχιστον 5 λιγότερες επεμβάσεις (τρέχον έτος)	52
II.μ	Ερώτημα 12: Απαιτούμενο προσωπικό ανά τμήμα, βάρδια, υποκλάση (εβδομάδα)	54
II.ν	Ερώτημα 13: Ιεραρχία εποπτείας ιατρού (Recursive CTE)	56
II.ξ	Ερώτημα 14: ICD-10 διαγνώσεις με ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη	58
II.ο	Ερώτημα 15: Κατανομή triage ανά επίπεδο επείγοντος	60
III.	Πηγές, Παραπομπές και Χρήση Εργαλείων Τεχνητής Νοημοσύνης	64
III.α	Επίσημες πηγές δεδομένων αναφοράς	64
III.β	Τεκμηρίωση και κοινότητες	64
III.γ	Χρήση Εργαλείων Τεχνητής Νοημοσύνης	65

I. Μέρος Α: Σχεδιασμός και Υλοποίηση Βάσης Δεδομένων

I.α ER Διάγραμμα

Στο παρακάτω σχήμα παρουσιάζεται το Εννοιολογικό Διάγραμμα Οντοτήτων-Συσχετίσεων (ER) για το νοσοκομείο «Υγειόπολις». Αναγνωρίστηκαν 20 κύριες οντότητες, οι οποίες καλύπτουν πλήρως τις απαιτήσεις της εκφώνησης.

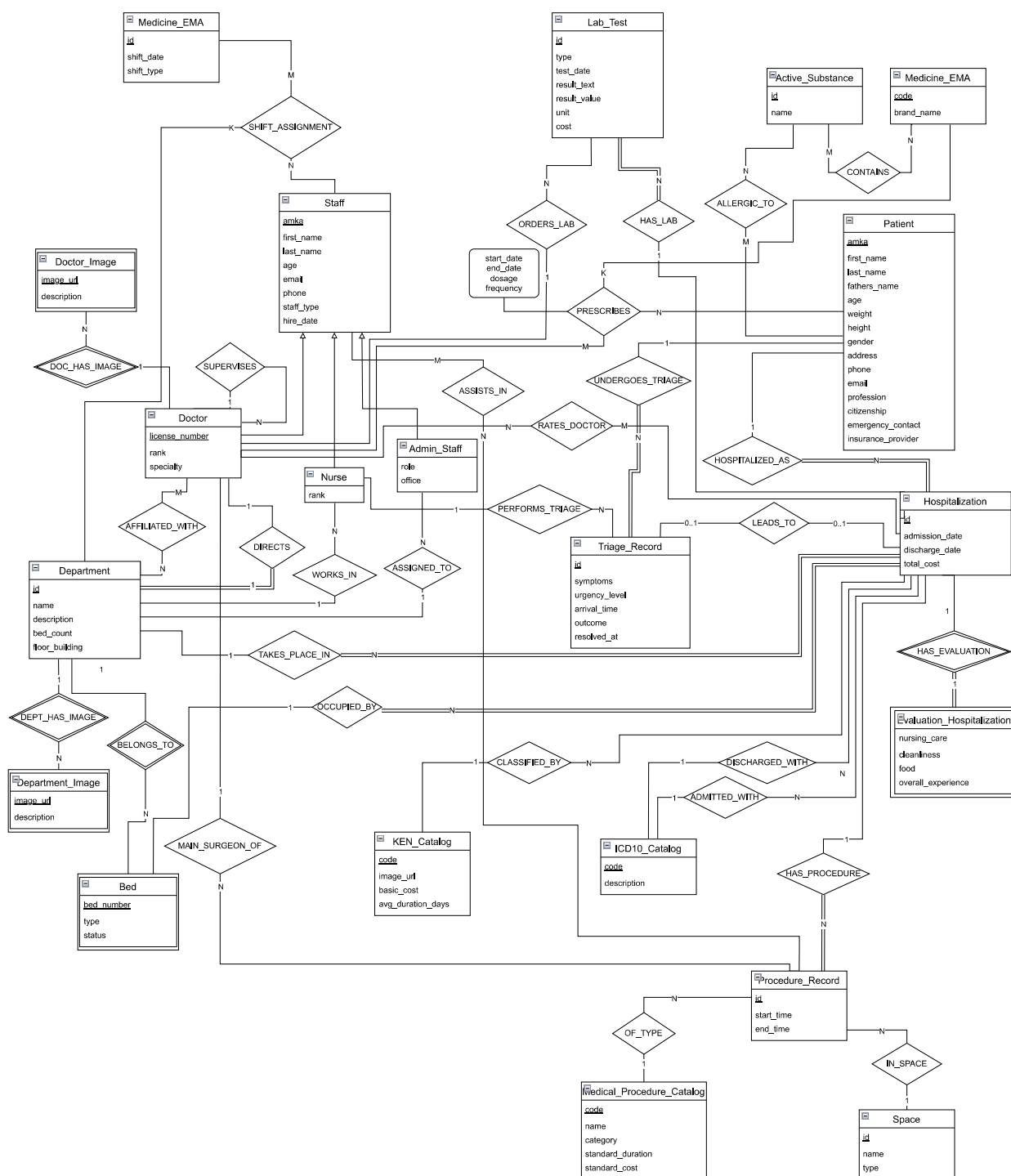


Figure 1: Διάγραμμα Οντοτήτων – Συσχετίσεων (ER)

Η μετάπτωση του εννοιολογικού μοντέλου στο σχεσιακό πραγματοποιήθηκε σε **MariaDB/MySQL**, με πλήρη κανονικοποίηση έως την 3η Κανονική Μορφή (3NF). Για την κληρονομικότητα του προσωπικού επιλέχθηκε *Class Table Inheritance*: κοινός πίνακας **Staff** με foreign key σε εξειδικευμένους πίνακες (**Doctors**, **Nurses**, **Admin_Staff**).



I.β.1 Σχεδιαστικές Πληροφορίες Πινάκων

Η βάση δεδομένων περιλαμβάνει **28 πίνακες** καταναμημένους σε 10 λειτουργικές κατηγορίες.

#	Πίνακας	#	Πίνακας
1	Staff	15	Procedure_Records
2	Doctors	16	Procedure_Assistants
3	Doctor_has_Department	17	Medicine_EMA
4	Nurses	18	Active_Substances
5	Admin_Staff	19	Medicine_has_Substances
6	Departments	20	Patient_Allergies
7	Beds	21	Prescriptions
8	Patients	22	Lab_Tests
9	Triage_Records	23	Evaluation_Hospitalization
10	ICD10_Catalog	24	Evaluation_Doctor
11	Hospitalization	25	Department_Images
12	KEN_Catalog	26	Doctor_Images
13	Medical_Procedure_Catalog	27	Shifts
14	Spaces	28	Shift_Assignments

Table 1: Πίνακες της Βάσης Δεδομένων *hygeiopolis_db*

I.β.1.1 Προσωπικό

Staff — Βασικός πίνακας για όλο το προσωπικό (ΑΜΚΑ, όνομα, επώνυμο, ηλικία, email, τηλέφωνο, ημερομηνία πρόσληψης, τύπος). Αποτελεί τη βάση της ιεραρχίας Class Table Inheritance.

Doctors — Εξειδίκευση για ιατρούς. Περιέχει αριθμό άδειας ιατρικού συλλόγου, ειδικότητα, βαθμίδα (Ειδικευόμενος, Επιμελητής Β΄, Επιμελητής Α΄, Διευθυντής) και αυτο-αναφορά *supervisor_amka* για την ιεραρχία εποπτείας.

Doctor_has_Department — Σχέση πολλά-προς-πολλά γιατρών-τμημάτων. Κάθε ιατρός μπορεί να ανήκει σε ένα ή περισσότερα τμήματα.

Nurses — Εξειδίκευση για νοσηλευτές. Περιέχει βαθμίδα (Βοηθός Νοσηλεύτη, Νοσηλεύτης, Προϊστάμενος) και FK στο τμήμα τους.

Admin_Staff — Εξειδίκευση για διοικητικό προσωπικό. Περιέχει ρόλο (Γραμματέας, Λογιστής κλπ.), γραφείο και FK στο τμήμα τους.

I.β.1.2 Τμήματα και Κλίνες

Departments — Αποθηκεύει τα 15 τμήματα του νοσοκομείου με όνομα, περιγραφή, αριθμό κλινών, όροφο/χτίριο και FK στον Διευθυντή τμήματος.

Beds — Κλίνες με μοναδικό αριθμό, τύπο (ΜΕΘ, Μονόκλινο, Πολύκλινο, Παιδιατρική) και κατάσταση (Διαθέσιμη, Κατειλημμένη, Υπό Συντήρηση). Κάθε κλίνη ανήκει σε ένα τμήμα.

I.β.1.3 Ασθενείς και Triage

Patients — Πλήρη δημογραφικά στοιχεία: AMKA, ονοματεπώνυμο, πατρώνυμο, ηλικία, φύλο, βάρος, ύψος, διεύθυνση, τηλέφωνο, email, επάγγελμα, υπηκοότητα, στοιχεία οικείου, ασφαλιστικός φορέας.

Triage_Records — Καταγράφει κάθε επίσκεψη ασθενή στα Επείγοντα. Περιλαμβάνει τον νοσηλευτή διαλογής, τα συμπτώματα, το επίπεδο επείγοντος (1-5), την ακριβή ώρα άφιξης για FIFO ταξινόμηση, καθώς και τα πεδία outcome (Admitted/Discharged/NULL), resolved_at και hospitalization_id για πλήρη ιχνηλάτηση του αποτελέσματος.

I.β.1.4 Νοσηλείες

Hospitalization — Κεντρικός πίνακας. Συνδέει ασθενή, κλίνη και τμήμα. Περιέχει ημερομηνίες εισαγωγής/εξόδου, διαγνώσεις ICD-10, κωδικό KEN και συνολικό κόστος. Το κόστος υπολογίζεται αυτόματα από trigger κατά το BEFORE UPDATE εξόδου.

ICD10_Catalog — Κατάλογος αναφοράς ICD-10 (~11.000 κωδικοί).

KEN_Catalog — Κλειστά Ενοποιημένα Νοσήλια (727 εγγραφές). Ορίζει βασικό κόστος και Μέση Διάρκεια Νοσηλείας (ΜΔΝ).

I.β.1.5 Ιατρικές Πράξεις

Medical_Procedure_Catalog — Κατάλογος ιατρικών πράξεων (~11.000 εγγραφές) με κωδικό, όνομα, κατηγορία (Χειρουργική / Διαγνωστική / Θεραπευτική), τυπική διάρκεια και κόστος.

Spaces — 10 χώροι (6 Χειρουργεία + 4 Αιθούσες Επεμβάσεων). Χρησιμοποιείται από τον trigger ελέγχου overlap.

Procedure_Records — Εγγραφές εκτελεσμένων πράξεων. Συνδέει νοσηλεία, κατάλογο, χώρο, κύριο χειρουργό και χρόνους έναρξης/λήξης.

Procedure_Assistants — Σχέση πολλά-προς-πολλά μεταξύ πράξεων και βοηθών (ιατροί ή νοσηλευτές).

I.β.1.6 Φάρμακα και Αλλεργίες

Medicine_EMA — Φάρμακα από τη βάση EMA Article 57 (~3.000 εγγραφές).

Active_Substances — Δραστικές ουσίες (1.211 εγγραφές από EMA).

Medicine_has_Substances — Σχέση φαρμάκων-δραστικών ουσιών.

Patient_Allergies — Αλλεργίες ασθενών σε δραστικές ουσίες. Χρησιμοποιείται από τον trigger anti-allergy κατά τη συνταγογράφηση.

Prescriptions — Συνταγογραφήσεις με σύνθετο PK (ιατρός, ασθενής, φάρμακο, ημ. έναρξης). Περιλαμβάνει δοσολογία, συχνότητα και ημ. λήξης.

I.β.1.7 Εργαστηριακές Εξετάσεις

Lab_Tests — Εξετάσεις (Αιματολογική, Βιοχημική, Απεικονιστική, Μικροβιολογική) συνδεδεμένες με νοσηλεία. Περιέχει ημερομηνία, αποτέλεσμα, κόστος και ιατρό-αιτούντα.

I.β.1.8 Αξιολογήσεις

Evaluation_Hospitalization — Αξιολόγηση νοσηλείας (κλίμακα Likert 1-5) σε 4 κριτήρια. Επιτρέπεται μόνο μετά ολοκλήρωσή της.

Evaluation_Doctor — Αξιολόγηση ιατρού (Ποιότητα ιατρικής φροντίδας) από ασθενή με ολοκληρωμένη νοσηλεία κατά την οποία ο ιατρός είχε συνταγογραφήσει.

I.β.1.9 Βάρδιες

Shifts — Ορίζει βάρδιες (ημερομηνία + τύπος: Morning/Afternoon/Night). Unique constraint `uq_shift_date_type` εξασφαλίζει μοναδικότητα.

Shift_Assignments — Σχέση πολλά-προς-πολλά βαρδιών, μελών προσωπικού και τμημάτων. Εξασφαλίζει (μέσω triggers) κάλυψη τουλάχιστον 3/6/2 ατόμων.

I.β.1.10 Εικόνες

Department_Images — Εικόνες τμημάτων με URL και λεκτική περιγραφή. FK με ON DELETE CASCADE στον πίνακα `Departments`.

Doctor_Images — Φωτογραφίες ιατρών με URL. FK με ON DELETE CASCADE στον πίνακα `Doctors`.

Σχεδιαστική επιλογή: Αντί γενικού πίνακα `Entity_Images`, επιλέχθηκαν δύο ξεχωριστοί πίνακες ανά οντότητα ώστε να δηλωθούν κανονικά Foreign Keys με ON DELETE CASCADE, κάτι που δεν επιτρέπεται με `polymorphic entity_type + entity_id` σχεδίαση.

I.β.2 Περιορισμοί Ορθότητας (Triggers)

Η βάση δεδομένων ενσωματώνει **22 triggers** (BEFORE/AFTER INSERT/UPDATE/DELETE) που επιβάλλουν αυτόματα όλους τους επιχειρησιακούς κανόνες. Κανένας κανόνας δεν μπορεί να παρακαμφθεί μέσω της εφαρμογής.

#	Trigger	Πίνακας	Γεγονός
1	check_allergy_before_prescription	Prescriptions	BEFORE INSERT
2	check_monthly_shift_limits	Shift_Assignments	BEFORE INSERT
3	check_shift_rest_and_night_limit	Shift_Assignments	BEFORE INSERT
4	check_doctor_supervision	Doctors	BEFORE INSERT
5	check_doctor_supervision_update	Doctors	BEFORE UP- DATE
6	calculate_hospitalization_cost	Hospitalization	BEFORE UP- DATE
7	check_resident_supervision_in_shift	Shift_Assignments	BEFORE INSERT
8	check_procedure_overlap	Procedure_Records	BEFORE INSERT
9	check_evaluation_hosp_completed	Eval. Hospitalization	BEFORE INSERT
10	check_evaluation_doctor_completed	Evaluation_Doctor	BEFORE INSERT
11	check_bed_department_match	Hospitalization	BEFORE INSERT
12	check_bed_availability	Hospitalization	BEFORE INSERT
13	check_discharge_after_admission	Hospitalization	BEFORE UP- DATE
14	check_prescription_during_hospitalization	Prescriptions	BEFORE INSERT
15	check_bed_availability_update	Hospitalization	BEFORE UP- DATE
16	check_bed_department_match_update	Hospitalization	BEFORE UP- DATE
17	check_procedure_overlap_update	Procedure_Records	BEFORE UP- DATE
18	auto_set_bed_occupied	Hospitalization	AFTER INSERT
19	auto_set_bed_available	Hospitalization	AFTER UPDATE
20	auto_calculate_cost_on_insert	Hospitalization	BEFORE INSERT
21	auto_calculate_cost_on_discharge	Hospitalization	BEFORE UP- DATE
22	check_minimum_shift_staffing_on_delete	Shift_Assignments	BEFORE DELETE

Table 2: Κατάλογος Triggers (22 συνολικά)

I.β.2.1 Ομάδα 1 – Συνταγογράφηση (Prescriptions)

Δύο triggers εξασφαλίζουν ορθότητα κάθε συνταγής.

Trigger #1: check_allergy_before_prescription — JOIN μεταξύ Medicine_has_Substances και Patient_Allergies: αν κάποια δραστική ουσία του φαρμάκου εμφανίζεται στις αλλεργίες του ασθενή, η συνταγογράφηση αποκλείεται.

```
1 CREATE TRIGGER check_allergy_before_prescription
2 BEFORE INSERT ON Prescriptions
3 FOR EACH ROW
4 BEGIN
5     DECLARE allergy_count INT;
6     SELECT COUNT(*) INTO allergy_count
7     FROM Medicine_has_Substances mhs
8     JOIN Patient_Allergies pa ON mhs.substance_id = pa.substance_id
9     WHERE mhs.medicine_code = NEW.medicine_code
10    AND pa.patient_amka = NEW.patient_amka;
11    IF allergy_count > 0 THEN
12        SIGNAL SQLSTATE '45000'
13        SET MESSAGE_TEXT = 'Αλλεργία: Ο ασθενής είναι αλλεργικός σε δρασ-
14        τική ουσία φαρμάκου.';
15    END IF;
16 END
```

Trigger #14: check_prescription_during_hospitalization — Η ημερομηνία έναρξης συνταγής πρέπει να εμπίπτει σε κάποια ενεργή ή ολοκληρωμένη νοσηλεία του ασθενή ($\text{admission_date} \leq \text{start_date} \leq \text{discharge_date}$).

```
1 CREATE TRIGGER check_prescription_during_hospitalization
2 BEFORE INSERT ON Prescriptions
3 FOR EACH ROW
4 BEGIN
5     IF NOT EXISTS (
6         SELECT 1 FROM Hospitalization
7         WHERE patient_amka = NEW.patient_amka
8             AND DATE(admission_date) <= NEW.start_date
9             AND (discharge_date IS NULL
10                OR DATE(discharge_date) >= NEW.start_date)
11     ) THEN
12         SIGNAL SQLSTATE '45000'
13         SET MESSAGE_TEXT = 'Σφάλμα: Συνταγογράφηση επιτρέπεται μόνο κατά
14         τη διάρκεια νοσηλείας.';
15     END IF;
16 END
```

I.β.2.2 Ομάδα 2 – Βάρδιες (Shift_Assignments)

Τέσσερις triggers καλύπτουν όλους τους κανόνες εφημεριών της εκφώνησης.

Trigger #2: check_monthly_shift_limits — Μέγιστα μηνιαία όρια: Ιατροί 15, Νοσηλευτές 20, Διοικητικοί 25.

```
1 CREATE TRIGGER check_monthly_shift_limits
2 BEFORE INSERT ON Shift_Assignments
3 FOR EACH ROW
4 BEGIN
5     DECLARE current_shifts INT;
6     DECLARE p_type VARCHAR(45);
7     DECLARE s_date DATE;
8     SELECT staff_type INTO p_type FROM Staff WHERE amka= NEW.staff_amka;
9     SELECT shift_date INTO s_date FROM Shifts WHERE id = NEW.shift_id;
10    SELECT COUNT(*) INTO current_shifts
11    FROM Shift_Assignments sa JOIN Shifts s ON sa.shift_id = s.id
12    WHERE sa.staff_amka = NEW.staff_amka
13        AND MONTH(s.shift_date) = MONTH(s_date)
14        AND YEAR(s.shift_date) = YEAR(s_date);
15    IF (p_type = 'Doctor' AND current_shifts >= 15) OR
16        (p_type = 'Nurse' AND current_shifts >= 20) OR
17        (p_type = 'Admin' AND current_shifts >= 25) THEN
18        SIGNAL SQLSTATE '45000'
19        SET MESSAGE_TEXT = 'Πέρβαση μέγιστου μηνιαίου ορίου βαρδιών για
20        την κατηγορία.';
21    END IF;
22 END
```

Trigger #3: check_shift_rest_and_night_limit — Συνδυάζει δύο ελέγχους: (α) max 3 συνεχόμενες νυχτερινές και (β) ελάχιστο δωρο ανάπαυσης μεταξύ διαδοχικών βαρδιών.

```
1 CREATE TRIGGER check_shift_rest_and_night_limit
2 BEFORE INSERT ON Shift_Assignments
3 FOR EACH ROW
4 BEGIN
5     DECLARE new_shift_type VARCHAR(20);
6     DECLARE new_shift_date DATE;
7     DECLARE night_streak INT DEFAULT 0;
8     SELECT shift_type, shift_date INTO new_shift_type, new_shift_date
9     FROM Shifts WHERE id = NEW.shift_id;
10    -- Ελεγχος max 3 συνεχόμενων νυχτερινών
11    IF new_shift_type = 'Night' THEN
12        SELECT COUNT(*) INTO night_streak
13        FROM Shift_Assignments sa JOIN Shifts s ON sa.shift_id = s.id
14        WHERE sa.staff_amka = NEW.staff_amka AND s.shift_type = 'Night'
15            AND s.shift_date IN (
16                DATE_SUB(new_shift_date, INTERVAL 1 DAY),
17                DATE_SUB(new_shift_date, INTERVAL 2 DAY));
18        IF night_streak >= 2 THEN
19            IF EXISTS (
20                SELECT 1 FROM Shift_Assignments sa2
21                JOIN Shifts s2 ON sa2.shift_id = s2.id
22                WHERE sa2.staff_amka = NEW.staff_amka
23                    AND s2.shift_type = 'Night'
24                    AND s2.shift_date = DATE_SUB(new_shift_date, INTERVAL
25                3 DAY)
26            ) THEN
27                SIGNAL SQLSTATE '45000'
```

```

27      SET MESSAGE_TEXT = 'Απαγόρευση: Δεν επιτρέπονται πάνω απ
ό 3 συνεχόμενες νυχτερινές.';
28      END IF;
29      END IF;
30      END IF;
31      -- Ελεγχος ελαχιστου θωρου αναπαυσης
32      IF EXISTS (
33          SELECT 1 FROM Shift_Assignments sa
34          JOIN Shifts s_ex ON sa.shift_id = s_ex.id
35          WHERE sa.staff_amka = NEW.staff_amka AND (
36              (s_ex.shift_date = new_shift_date AND (
37                  (s_ex.shift_type='Morning' AND new_shift_type='
Afternoon')
38                  OR (s_ex.shift_type='Afternoon' AND new_shift_type='Morning'
39                  OR (s_ex.shift_type='Afternoon' AND new_shift_type='Night')
40                  OR (s_ex.shift_type='Night' AND new_shift_type='
Afternoon'))))
41              OR (DATEDIFF(new_shift_date,s_ex.shift_date)=1
42                  AND s_ex.shift_type='Night' AND new_shift_type='Morning'
43              )
44              OR (DATEDIFF(s_ex.shift_date,new_shift_date)=1
45                  AND new_shift_type='Night' AND s_ex.shift_type='Morning
46              ))
47          ) THEN
48      SIGNAL SQLSTATE '45000'
49      SET MESSAGE_TEXT = 'Απαγόρευση: Απαιτείται τουλάχιστον θωρο ανάπ
αυσης μεταξύ βαρδιών.';
50      END IF;
51      END

```

Trigger #7: check_resident_supervision_in_shift — Αν ο νέος εργαζόμενος είναι Ειδικευόμενος, ελέγχει αν υπάρχει ήδη Επιμελητής Α΄ ή Διευθυντής στην ίδια βάρδια και τμήμα.

```

1  CREATE TRIGGER check_resident_supervision_in_shift
2  BEFORE INSERT ON Shift_Assignments
3  FOR EACH ROW
4  BEGIN
5      DECLARE is_resident      INT DEFAULT 0;
6      DECLARE senior_present   INT DEFAULT 0;
7      SELECT COUNT(*) INTO is_resident FROM Doctors
8      WHERE staff_amka = NEW.staff_amka AND 'rank' = 'Ειδικευόμενος';
9      IF is_resident > 0 THEN
10         SELECT COUNT(*) INTO senior_present
11         FROM Shift_Assignments sa JOIN Doctors d ON sa.staff_amka = d.
staff_amka
12         WHERE sa.shift_id      = NEW.shift_id
13             AND sa.department_id = NEW.department_id
14             AND d.'rank' IN ('Επιμελητης Α'', 'Διευθυντης');
15         IF senior_present = 0 THEN
16             SIGNAL SQLSTATE '45000'
17             SET MESSAGE_TEXT =
18                 'Σφάλμα: βάρδια με Ειδικευομενο απαιτει Επιμελητη Α ή Διευ
19                 θυντη.';
20         END IF;
21     END IF;
22     END

```

Trigger #22: check_minimum_shift_staffing_on_delete — Εμποδίζει διαγραφή ανάθεσης βάρδιας αν θα αφήσει τη βάρδια κάτω από τα minimums (3/6/2). Μετράει τι θα απομείνει μετά τη διαγραφή, εξαιρώντας την τρέχουσα εγγραφή (staff_amka <> OLD.staff_amka).

```
1 CREATE TRIGGER check_minimum_shift_staffing_on_delete
2 BEFORE DELETE ON Shift_Assignments
3 FOR EACH ROW
4 BEGIN
5     DECLARE doc_count INT DEFAULT 0;
6     DECLARE nur_count INT DEFAULT 0;
7     DECLARE adm_count INT DEFAULT 0;
8     SELECT
9         SUM(CASE WHEN s.staff_type = 'Doctor' THEN 1 ELSE 0 END),
10        SUM(CASE WHEN s.staff_type = 'Nurse' THEN 1 ELSE 0 END),
11        SUM(CASE WHEN s.staff_type = 'Admin' THEN 1 ELSE 0 END)
12 INTO doc_count, nur_count, adm_count
13 FROM Shift_Assignments sa JOIN Staff s ON sa.staff_amka = s.amka
14 WHERE sa.shift_id = OLD.shift_id
15        AND sa.department_id = OLD.department_id
16        AND sa.staff_amka <> OLD.staff_amka;
17 IF doc_count < 3 THEN SIGNAL SQLSTATE '45000'
18     SET MESSAGE_TEXT = 'Απαγορευση: < 3 ιατροι στη βαρδια.'; END IF;
19 IF nur_count < 6 THEN SIGNAL SQLSTATE '45000'
20     SET MESSAGE_TEXT = 'Απαγορευση: < 6 νοσηλευτες στη βαρδια.'; END
21 IF;
22 IF adm_count < 2 THEN SIGNAL SQLSTATE '45000'
23     SET MESSAGE_TEXT = 'Απαγορευση: < 2 διοικητικοι στη βαρδια.';
24 END IF;
```

I.β.2.3 Ομάδα 3 – Εποπτεία Ιατρών (Doctors)

Triggers #4 και #5: check_doctor_supervision / check_doctor_supervision_update — Ίδιοι κανόνες για INSERT και UPDATE: Ειδικευόμενος χωρίς επόπτη → SIGNAL, Διευθυντής με επόπτη → SIGNAL, άμεσος κύκλος εποπτείας βάθους 1 → SIGNAL. Βαθύτεροι κύκλοι ανιχνεύονται από assert_no_supervisor_cycles().

```
1 CREATE TRIGGER check_doctor_supervision
2 BEFORE INSERT ON Doctors
3 FOR EACH ROW
4 BEGIN
5     IF NEW.rank = 'Ειδικευόμενος' AND NEW.supervisor_amka IS NULL THEN
6         SIGNAL SQLSTATE '45000'
7         SET MESSAGE_TEXT = 'Σφάλμα: Οι ειδικευόμενοι ιατροί πρέπει υποχρ
8 εωτικά να έχουν επόπτη.';
9     END IF;
10    IF NEW.rank = 'Διευθυντής' AND NEW.supervisor_amka IS NOT NULL THEN
11        SIGNAL SQLSTATE '45000'
12        SET MESSAGE_TEXT = 'Σφάλμα: Οι Διευθυντές δεν επιτρέπεται να έχο
13 υν επόπτη.';
14    END IF;
15    IF NEW.supervisor_amka IS NOT NULL THEN
16        IF EXISTS (
17            SELECT 1 FROM Doctors
18            WHERE staff_amka = NEW.supervisor_amka
19              AND supervisor_amka = NEW.staff_amka
20        ) THEN
21            SIGNAL SQLSTATE '45000'
22            SET MESSAGE_TEXT = 'Σφάλμα: Κυκλική αλυσίδα εποπτείας απαγορ
23 εύεται.';
24        END IF;
25    END IF;
26 END
```

I.β.2.4 Ομάδα 4 – Νοσηλείες: Ακεραιότητα INSERT (Hospitalization)

Trigger #11: check_bed_department_match — Η κλίνη πρέπει να ανήκει στο τμήμα της νοσηλείας (JOIN Beds).

Trigger #12: check_bed_availability — Η κλίνη δεν πρέπει να έχει άλλη ενεργή νοσηλεία (discharge_date IS NULL).

```
1 CREATE TRIGGER check_bed_department_match
2 BEFORE INSERT ON Hospitalization
3 FOR EACH ROW
4 BEGIN
5     IF NOT EXISTS (SELECT 1 FROM Beds
6                     WHERE id = NEW.bed_id AND department_id = NEW.
7                     department_id)
8     THEN
9         SIGNAL SQLSTATE '45000'
10        SET MESSAGE_TEXT = 'Σφάλμα: κλινη δεν ανηκει στο τμημα.';
11    END IF;
12 END
13
14 CREATE TRIGGER check_bed_availability
15 BEFORE INSERT ON Hospitalization
16 FOR EACH ROW
17 BEGIN
18     IF EXISTS (SELECT 1 FROM Hospitalization
19                WHERE bed_id = NEW.bed_id AND discharge_date IS NULL)
20    THEN
21        SIGNAL SQLSTATE '45000'
22        SET MESSAGE_TEXT = 'Σφάλμα: κλινη ηδη κατειλημμενη.';
23    END IF;
24 END
```

I.β.2.5 Ομάδα 5 – Νοσηλείες: Ακεραιότητα UPDATE + Αυτοματισμοί

Trigger #13: check_discharge_after_admission — discharge_date > admission_date.

Triggers #15/#16 — Αντικατοπτρίζουν #11/#12 για UPDATE (με id <> NEW.id για self-exclusion).

Triggers #18/#19: auto_set_bed_occupied/available — Συγχρονίζουν αυτόματα Beds.status κατά INSERT/UPDATE νοσηλείας.

```
1 CREATE TRIGGER auto_set_bed_occupied
2 AFTER INSERT ON Hospitalization
3 FOR EACH ROW
4 BEGIN
5     IF NEW.discharge_date IS NULL THEN
6         UPDATE Beds SET status = 'Κατειλημμενη' WHERE id = NEW.bed_id;
7     END IF;
8 END
9
10 CREATE TRIGGER auto_set_bed_available
11 AFTER UPDATE ON Hospitalization
12 FOR EACH ROW
13 BEGIN
14     IF OLD.discharge_date IS NULL AND NEW.discharge_date IS NOT NULL
15     THEN
16         UPDATE Beds SET status = 'Διαθεσιμη' WHERE id = NEW.bed_id;
17     END IF;
18 END
```

Triggers #20/#21: auto_calculate_cost_on_insert/discharge — Υπολογίζουν αυτόματα total_cost: βασικό κόστος KEN + max(0, actual_days – MDN) × 100€.

```
1 CREATE TRIGGER auto_calculate_cost_on_insert
2 BEFORE INSERT ON Hospitalization
3 FOR EACH ROW
4 BEGIN
5     IF NEW.discharge_date IS NOT NULL THEN
6         SET NEW.total_cost = (
7             SELECT k.basic_cost + GREATEST(0,
8                 DATEDIFF(NEW.discharge_date, NEW.admission_date)
9                 - k.avg_duration_days) * 100.00
10            FROM KEN_Catalog k WHERE k.code = NEW.ken_code);
11     ELSE
12         SET NEW.total_cost = (
13             SELECT k.basic_cost FROM KEN_Catalog k
14             WHERE k.code = NEW.ken_code);
15     END IF;
16 END
```

I.β.2.6 Ομάδα 6 – Overlap Επεμβάσεων (Procedure_Records)

Triggers #8 και #17: Αποτρέπουν ταυτόχρονη χρήση χώρου ή χειρουργού. Ο #17 (UPDATE) προσθέτει id <> NEW.id για self-exclusion.

```
1 CREATE TRIGGER check_procedure_overlap
2 BEFORE INSERT ON Procedure_Records
3 FOR EACH ROW
4 BEGIN
5     IF EXISTS (SELECT 1 FROM Procedure_Records
6                 WHERE space_id = NEW.space_id
7                       AND start_time < NEW.end_time
8                       AND end_time > NEW.start_time) THEN
9         SIGNAL SQLSTATE '45000'
10        SET MESSAGE_TEXT = 'Σφάλμα: ο χώρος είναι ήδη κατειλημμένος.';
11    END IF;
12    IF EXISTS (SELECT 1 FROM Procedure_Records
13                WHERE main_surgeon_amk = NEW.main_surgeon_amk
14                      AND start_time < NEW.end_time
15                      AND end_time > NEW.start_time) THEN
16        SIGNAL SQLSTATE '45000'
17        SET MESSAGE_TEXT = 'Σφάλμα: ο ιατρός συμμετεχει ήδη σε επεμβαση.';
18    END IF;
19 END
```


I.β.2.7 Ομάδα 7 – Αξιολογήσεις

Triggers #9 και #10: Αξιολόγηση νοσηλείας/ιατρού μόνο αν discharge_date IS NOT NULL.

```
1 CREATE TRIGGER check_evaluation_hosp_completed
2 BEFORE INSERT ON Evaluation_Hospitalization
3 FOR EACH ROW
4 BEGIN
5     IF NOT EXISTS (SELECT 1 FROM Hospitalization
6                     WHERE id = NEW.hospitalization_id
7                     AND discharge_date IS NOT NULL) THEN
8         SIGNAL SQLSTATE '45000'
9         SET MESSAGE_TEXT =
10             'Σφάλμα: αξιολογηση μονο μετα την εξοδο.';
11     END IF;
12 END
```

I.β.3 Stored Procedures και Functions

Αναπτύχθηκαν **3 functions** και **7 stored procedures**.

#	Όνομα
	Περιγραφή
<i>Functions</i>	
1	calculate_hospitalization_cost(p_hosp_id)
	Επιστρέφει: <code>basic_cost + max(0, actual_days - MDN) × 100€</code> . Χρησιμοποιείται από <code>discharge_patient()</code> και <code>recalculate_all_hospitalization_costs()</code> .
2	get_department_revenue(p_dept_id, p_year)
	SUM(<code>total_cost</code>) για νοσηλείες τμήματος συγκεκριμένου έτους.
3	available_beds_in_dept(p_dept_id)
	Κλίνες που δεν είναι «Υπό Συντήρηση» ΚΑΙ δεν έχουν ενεργή νοσηλεία.
<i>Procedures</i>	
4	admit_patient(p_pat_amka, p_dept_id, p_admission_diagnosis_icd10, p_ken_code, p_new_hosp_id)
	Atomic εισαγωγή: εύρεση πρώτης διαθέσιμης κλίνης, INSERT νοσηλείας UPDATE κλίνης → «Κατειλημμένη». Χρήση START TRANSACTION.
5	discharge_patient(hosp_id, date, icd10)
	Atomic έξοδος: UPDATE discharge fields, τελικό κόστος μέσω function, UPDATE κλίνης → «Διαθέσιμη».
6	assign_to_shift(shift_id, amka, dept_id)
	Transactional wrapper: ένα INSERT που ενεργοποιεί αυτόματα όλους τους shift triggers (#2, #3, #7, #22).
7	get_patient_history(p_patient_amka)
	Multi-result-set: νοσηλείες (με τμήμα, ICD, κόστος), συνταγές (με φάρμακο, ιατρό), εξετάσεις — χρονολογικά.
8	get_doctor_workload(p_doctor_amka, p_year)
	Βάρδιες, κύριες επεμβάσεις, βοηθητικές, συνταγές, εξετάσεις ιατρού για συγκεκριμένο έτος — single result set.
9	recalculate_all_hospitalization_costs()
	UPDATE Hospitalization SET <code>total_cost</code> = <code>calculate_hospitalization_cost(id)</code> για όλες τις ολοκληρωμένες νοσηλείες.
10	assert_no_supervisor_cycles()
	Recursive CTE: εντοπίζει κύκλους εποπτείας βάθους ≥ 3 (χρήσιμο μετά bulk load με FOREIGN_KEY_CHECKS=0).

Table 3: Stored Procedures & Functions

I.β.3.1 Υλοποίηση

1. calculate_hospitalization_cost(p_hosp_id)

```
1 CREATE FUNCTION calculate_hospitalization_cost(p_hosp_id INT)
2 RETURNS DECIMAL(10,2)
3 READS SQL DATA
4 DETERMINISTIC
5 BEGIN
6     DECLARE v_basic      DECIMAL(10,2) DEFAULT 0;
7     DECLARE v_mdn        INT           DEFAULT 0;
8     DECLARE v_actual     INT           DEFAULT 0;
9     DECLARE v_extra_per_day DECIMAL(10,2) DEFAULT 100.00;
10    DECLARE v_total      DECIMAL(10,2) DEFAULT 0;
11    DECLARE v_adm DATETIME;
12    DECLARE v_dis DATETIME;
13
14    SELECT k.basic_cost, k.avg_duration_days, h.admission_date, h.
15    discharge_date
16    INTO v_basic, v_mdn, v_adm, v_dis
17    FROM Hospitalization h
18    JOIN KEN_Catalog k ON h.ken_code = k.code
19    WHERE h.id = p_hosp_id;
20
21    IF v_dis IS NULL THEN
22        -- Ενεργή νοσηλεία: επιστρέφουμε basic cost μόνο
23        RETURN v_basic;
24    END IF;
25
26    SET v_actual = DATEDIFF(v_dis, v_adm);
27    SET v_total = v_basic + GREATEST(0, v_actual - v_mdn) *
28    v_extra_per_day;
29    RETURN v_total;
30 END
```

2. get_department_revenue(p_dept_id INT, p_year INT)

```
1 CREATE FUNCTION get_department_revenue(p_dept_id INT, p_year INT)
2 RETURNS DECIMAL(12,2)
3 READS SQL DATA
4 DETERMINISTIC
5 BEGIN
6     DECLARE v_revenue DECIMAL(12,2) DEFAULT 0;
7
8     SELECT COALESCE(SUM(total_cost), 0) INTO v_revenue
9     FROM Hospitalization
10    WHERE department_id = p_dept_id
11        AND YEAR(admission_date) = p_year;
12
13    RETURN v_revenue;
14 END
```

3. available_beds_in_dept(p_dept_id INT)

```
1 CREATE FUNCTION available_beds_in_dept(p_dept_id INT)
2 RETURNS INT
3 READS SQL DATA
4 DETERMINISTIC
5 BEGIN
6     DECLARE v_count INT DEFAULT 0;
7     SELECT COUNT(*) INTO v_count
8     FROM Beds b
9     WHERE b.department_id = p_dept_id
10         AND b.status <> 'Τό Συντήρηση'
11         AND NOT EXISTS (
12             SELECT 1 FROM Hospitalization h
13             WHERE h.bed_id = b.id
14                 AND h.discharge_date IS NULL
15         );
16     RETURN v_count;
17 END
```

4. admit_patient(..)

```
1 CREATE PROCEDURE admit_patient(
2     IN p_patient_amka VARCHAR(45),
3     IN p_department_id INT,
4     IN p_admission_diagnosis_icd10 VARCHAR(10),
5     IN p_ken_code VARCHAR(20),
6     OUT p_new_hosp_id INT
7 )
8 BEGIN
9     DECLARE v_bed_id INT DEFAULT NULL;
10    DECLARE v_basic_cost DECIMAL(10,2) DEFAULT 0;
11
12    DECLARE EXIT HANDLER FOR SQLEXCEPTION
13    BEGIN
14        ROLLBACK;
15        RESIGNAL;
16    END;
17
18    START TRANSACTION;
19
20    -- Βρίσκουμε την πρώτη διαθέσιμη κλίνη
21    SELECT b.id INTO v_bed_id
22    FROM Beds b
23    WHERE b.department_id = p_department_id
24        AND b.status = 'Διαθέσιμη'
25        AND NOT EXISTS (
26            SELECT 1 FROM Hospitalization h
27            WHERE h.bed_id = b.id AND h.discharge_date IS NULL
28        )
29    LIMIT 1;
30
31    IF v_bed_id IS NULL THEN
32        SIGNAL SQLSTATE '45000'
33        SET MESSAGE_TEXT = 'Σφάλμα: Δεν υπάρχει διαθέσιμη κλίνη στο
34        τμήμα.';
35    END IF;
36
37    -- Βασικό κόστος από KEN
```

```

37 SELECT basic_cost INTO v_basic_cost
38 FROM KEN_Catalog WHERE code = p_ken_code;
39
40 -- Δημιουργία νοσηλείας
41 INSERT INTO Hospitalization
42 (patient_amka, bed_id, department_id, admission_date,
43 admission_diagnosis_icd10, ken_code, total_cost)
44 VALUES
45 (p_patient_amka, v_bed_id, p_department_id, NOW(),
46 p_admission_diagnosis_icd10, p_ken_code, v_basic_cost);
47
48 SET p_new_hosp_id = LAST_INSERT_ID();
49
50 -- Κλίνη Κατειλημμένη
51 UPDATE Beds SET status = 'Κατειλημμένη' WHERE id = v_bed_id;
52
53 COMMIT;
54 END

```

5. discharge_patient(hosp_id, date, icd)

```

1 CREATE PROCEDURE discharge_patient(
2     IN p_hosp_id INT,
3     IN p_discharge_date DATETIME,
4     IN p_discharge_diagnosis_icd10 VARCHAR(10)
5 )
6 BEGIN
7     DECLARE v_bed_id INT;
8     DECLARE v_final_cost DECIMAL(10,2);
9     DECLARE v_already_discharged DATETIME;
10
11     DECLARE EXIT HANDLER FOR SQLEXCEPTION
12     BEGIN
13         ROLLBACK;
14         RESIGNAL;
15     END;
16
17     START TRANSACTION;
18
19     SELECT bed_id, discharge_date INTO v_bed_id,
20 v_already_discharged
21 FROM Hospitalization WHERE id = p_hosp_id;
22
23 IF v_already_discharged IS NOT NULL THEN
24     SIGNAL SQLSTATE '45000'
25     SET MESSAGE_TEXT = 'Σφάλμα: Η νοσηλεία είναι ήδη κλεισμένη.';
26 END IF;
27
28 -- Ενημέρωση discharge fields
29 UPDATE Hospitalization
30 SET discharge_date = p_discharge_date,
31     discharge_diagnosis_icd10 = p_discharge_diagnosis_icd10
32 WHERE id = p_hosp_id;
33
34 -- Υπολογισμός & αποθήκευση τελικού κόστους
35 SET v_final_cost = calculate_hospitalization_cost(p_hosp_id);
36 UPDATE Hospitalization

```

```

36     SET total_cost = v_final_cost
37     WHERE id = p_hosp_id;
38
39     -- Απελευθέρωση κλίνης
40     UPDATE Beds SET status = 'Διαθέσιμη' WHERE id = v_bed_id;
41
42     COMMIT;
43 END

```

6. assign_to_shift(p_shift_id, p_staff_amka, p_department_id)

```

1 CREATE PROCEDURE assign_to_shift(
2     IN p_shift_id INT,
3     IN p_staff_amka VARCHAR(45),
4     IN p_department_id INT
5 )
6 BEGIN
7     DECLARE EXIT HANDLER FOR SQLEXCEPTION
8     BEGIN
9         ROLLBACK;
10        RESIGNAL;
11    END;
12
13    START TRANSACTION;
14    INSERT INTO Shift_Assignments (shift_id, staff_amka,
15    department_id)
16    VALUES (p_shift_id, p_staff_amka, p_department_id);
17    COMMIT;
18 END

```

7. get_patient_history(p_patient_amka)

```

1 CREATE PROCEDURE get_patient_history(IN p_patient_amka VARCHAR(45))
2 BEGIN
3     -- Νοσηλείες
4     SELECT
5         h.id, h.admission_date, h.discharge_date,
6         d.name AS department,
7         h.admission_diagnosis_icd10,
8         h.discharge_diagnosis_icd10,
9         h.ken_code, h.total_cost
10    FROM Hospitalization h
11    JOIN Departments d ON h.department_id = d.id
12    WHERE h.patient_amka = p_patient_amka
13    ORDER BY h.admission_date DESC;
14
15    -- Συνταγές
16    SELECT
17        p.medicine_code,
18        m.brand_name,
19        p.start_date, p.end_date,
20        p.dosage, p.frequency,
21        CONCAT(s.first_name, ' ', s.last_name) AS doctor
22    FROM Prescriptions p
23    JOIN Medicine_EMA m ON p.medicine_code = m.code
24    JOIN Staff s ON p.doctor_amka = s.amka
25    WHERE p.patient_amka = p_patient_amka
26    ORDER BY p.start_date DESC;

```

```

27
28      -- Εξετάσεις (μέσω νοσηλείων)
29      SELECT
30          l.type, l.test_date, l.result_text,
31          l.result_value, l.unit, l.cost
32      FROM Lab_Tests l
33      JOIN Hospitalization h ON l.hospitalization_id = h.id
34      WHERE h.patient_amka = p_patient_amka
35      ORDER BY l.test_date DESC;
36  END

```

8. get_doctor_workload(p_doctor_amka, p_year)

```

1  CREATE PROCEDURE get_doctor_workload(
2      IN p_doctor_amka VARCHAR(45),
3      IN p_year INT
4  )
5  BEGIN
6      SELECT
7          (SELECT COUNT(*)
8           FROM Shift_Assignments sa
9           JOIN Shifts sh ON sa.shift_id = sh.id
10          WHERE sa.staff_amka = p_doctor_amka
11                AND YEAR(sh.shift_date) = p_year)          AS total_shifts
12      ,
13          (SELECT COUNT(*)
14           FROM Procedure_Records pr
15           WHERE pr.main_surgeon_amk = p_doctor_amka
16                AND YEAR(pr.start_time) = p_year)          AS
17      lead_surgeries ,
18          (SELECT COUNT(*)
19           FROM Procedure_Assistants pa
20           JOIN Procedure_Records pr ON pa.procedure_record_id = pr.
21      id
22           WHERE pa.staff_amka = p_doctor_amka
23                AND YEAR(pr.start_time) = p_year)          AS
24      assisted_surgeries ,
25          (SELECT COUNT(*)
26           FROM Prescriptions p
27           WHERE p.doctor_amka = p_doctor_amka
28                AND YEAR(p.start_date) = p_year)          AS
29      prescriptions_issued ,
30          (SELECT COUNT(*)
31           FROM Lab_Tests l
32           WHERE l.ordering_doctor_amka = p_doctor_amka
33                AND YEAR(l.test_date) = p_year)          AS
34      lab_tests_ordered;
35  END

```

9. recalculate_all_hospitalization_costs()

```
1 CREATE PROCEDURE recalculate_all_hospitalization_costs()
2 BEGIN
3     UPDATE Hospitalization h
4     SET h.total_cost = calculate_hospitalization_cost(h.id)
5     WHERE h.discharge_date IS NOT NULL;
6
7     SELECT ROW_COUNT() AS rows_updated;
8 END
```

10. assert_no_supervisor_cycles()

```
1 CREATE PROCEDURE assert_no_supervisor_cycles()
2 BEGIN
3     DECLARE bad_count INT DEFAULT 0;
4
5     WITH RECURSIVE chain (root_amka, next_amka, depth, visited,
6 is_cycle) AS (
7         SELECT staff_amka,
8                 supervisor_amka,
9                 1,
10                CAST(CONCAT(',', staff_amka, ',' ) AS CHAR(2000)),
11                0
12         FROM Doctors
13
14         UNION ALL
15
16         SELECT c.root_amka,
17                d.supervisor_amka,
18                c.depth + 1,
19                CONCAT(c.visited, c.next_amka, ',' ),
20                CASE
21                    WHEN LOCATE(CONCAT(',', c.next_amka, ',' ), c.
22 visited) > 0 THEN 1
23                    ELSE 0
24                END
25         FROM chain c
26         JOIN Doctors d ON d.staff_amka = c.next_amka
27         WHERE c.next_amka IS NOT NULL
28                AND c.is_cycle = 0
29                AND c.depth < 50
30     )
31     SELECT COUNT(*) INTO bad_count
32     FROM chain
33     WHERE is_cycle = 1;
34
35     IF bad_count > 0 THEN
36         SIGNAL SQLSTATE '45000'
37         SET MESSAGE_TEXT = 'Σφάλμα: Εντοπίστηκε κύκλος στην αλυσίδα
38 εποπτείας ιατρών.';
39     END IF;
40 END
```


I.β.3.2 Παραδείγματα χρήσης

```
1  -- Atomic εισαγωγή ασθενή (βρίσκει κλίνη αυτοματα):
2  CALL admit_patient('12345678901', 1, 'I21', 'KEN042', @id);
3  SELECT @id;    -- επιστρέφει το νέο hospitalization_id
4
5  -- Εξοδος ασθενή με τελική διαγνώση:
6  CALL discharge_patient(@id, NOW(), 'I50');
7
8  -- Ιστορικό ασθενή (3 result sets: νοσηλίες, συνταγές, εξετάσεις):
9  CALL get_patient_history('12345678901');
10
11 -- Φορτος εργασίας ιατρού για 2026:
12 CALL get_doctor_workload('09876543210', 2026);
13
14 -- Εσοδα Καρδιολογίας (id=1) για 2026:
15 SELECT get_department_revenue(1, 2026);
16
17 -- Διαθεσιμες κλίνες ΜΕΘ (id=3):
18 SELECT available_beds_in_dept(3);
19
20 -- Έλεγχος κυκλικής εποπτείας μετά bulk load:
21 CALL assert_no_supervisor_cycles();
```

Μεταφορά ασθενή από Triage σε Νοσηλεία

Μια μεγάλη δυνατότητα της βάσης μας είναι η μεταφορά ασθενή από Triage σε Νοσηλεία, Για να γίνει αυτό καλούμε την διαδικασία `admit_patient(..)` και στη συνέχεια ενημερώνουμε τον πίνακα διαλογής αντίστοιχα. Η υλοποίηση και η ροή φαίνεται στο παρακάτω κομμάτι κώδικα.

```
1  -- Βήμα 1: Εισαγωγή ασθενή σε νοσηλεία (atomic procedure)
2  -- Triggers που ενεργοποιούνται αυτόματα:
3  -- #11 check_bed_department_match    -> κλίνη ανήκει στο τμήμα
4  -- #12 check_bed_availability        -> κλίνη είναι ελεύθερη
5  -- #20 auto_calculate_cost_on_insert -> total_cost = KEN basic_cost
6  -- #18 auto_set_bed_occupied         -> Beds.status = 'Κατειλημμένη'
7  CALL admit_patient(
8      '44417827823',    -- patient_amka
9      3,                -- department_id (πχ ΜΕΘ)
10     'I63',             -- admission_diagnosis_icd10
11     'B76A',            -- ken_code
12     @new_hosp_id       -- OUT: το id της νέας νοσηλείας
13 );
14
15 -- Βήμα 2: Ενημέρωση του triage record ως resolved
16 UPDATE Triage_Records
17     SET outcome          = 'Admitted',
18         resolved_at      = NOW(),
19         hospitalization_id = @new_hosp_id
20 WHERE id = 786          -- triage_id
21     AND outcome IS NULL; -- ασφαλής έλεγχος: δεν ξαναεπεξεργαζόμαστε
22
23 -- Αποτέλεσμα:
24 -- - Triage #786: outcome='Admitted', εξαφανίζεται από την queue
25 -- - Hospitalization @new_hosp_id: ενεργή, κόστος = KEN basic_cost
26 -- - Κλίνη: status='Κατειλημμένη' (αυτόματα από trigger #18)
```

I.β.4 Ευρετήρια (Indexes)

Η βάση δεδομένων MariaDB/MySQL δημιουργεί αυτόματα πολλαπλά indexes με βάση το primary key του κάθε πίνακα και τα foreign key που χρησιμοποιεί ο κάθε πίνακας. Γι' αυτό δεν χρειάστηκε να φτιάξουμε επιπρόσθετα Indexes για αυτά τα πεδία. Ωστόσο προσθέσαμε indexes σε πεδία όπου χρησιμοποιούνται από τα ερωτήματα (queries) ώστε να γίνεται γρηγορότερη η πρόσβαση στα πεδία αυτά.

Ευρετήριο	Πίνακας / Στήλη(ες)	Αιτιολόγηση
idx_hosp_admission_year	Hospitalization(admission_date)	Q1, Q9, Q14: YEAR(admission_date) range scan.
idx_hosp_dept_year	Hospitalization(department_id, admission_date)	Q1: σύνθετο φίλτρο dept × year — covering index.
idx_proc_start_time	Procedure_Records(start_time)	Q11: YEAR(start_time) — αποφεύγει full scan.
idx_triage_arrival	Triage_Records(arrival_time, urgency_level)	Q15: LEFT JOIN με παράθυρο 24ωρου.
idx_pres_patient_start	Prescriptions(patient_amka, start_date)	Q10: φίλτρο κατά patient ΚΑΙ start_date.
idx_lab_test_date	Lab_Tests(test_date)	Χρονολογικές αναφορές.

Table 4: Ευρετήρια Βάσης Δεδομένων με Αιτιολόγηση

II. Μέρος B: Ερωτήματα SQL

Σε αυτήν την ενότητα αναλύουμε και παρουσιάζουμε τα ζητούμενα ερωτήματα SQL 1 - 15. Τα αποτελέσματα αποτελούν στιγμιότυπα οθόνης από το webapp που αναπτύξαμε.

II.α Ερώτημα 1: Συνολικά έσοδα ανά τμήμα και ανά έτος

II.α.1 Ζητούμενο

Βρείτε τα συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό (βασικό κόστος έναντι πρόσθετης χρέωσης λόγω υπέρβασης ΜΔΝ) και κατανομή νοσηλειών ανά ασφαλιστικό φορέα.

II.α.2 Εξήγηση

Υπολογίζονται βασικά έσοδα (KEN) και πρόσθετα λόγω υπέρβασης ΜΔΝ, ανά τμήμα, έτος, κωδικό KEN και ασφαλιστικό φορέα. Το `GREATEST(0, ...)` εξασφαλίζει μη αρνητική πρόσθετη χρέωση (νοσηλεία < ΜΔΝ). Φιλτράρονται μόνο ολοκληρωμένες νοσηλείες (`discharge_date IS NOT NULL`).

II.α.3 Υλοποίηση

```
1 SELECT
2     d.name                                AS Department ,
3     YEAR(h.admission_date)               AS Admission_Year ,
4     h.ken_code                           AS KEN_Code ,
5     SUM(k.basic_cost)                    AS Total_Base_Revenue
6 ,
7     SUM(GREATEST(0, h.total_cost - k.basic_cost)) AS
8     Total_Extra_Revenue ,
9     p.insurance_provider                  AS Insurance_Provider
10 ,
11     COUNT(h.id)                          AS
12     Total_Hospitalizations
13 FROM Hospitalization h
14 JOIN Departments d ON h.department_id = d.id
15 JOIN KEN_Catalog k ON h.ken_code = k.code
16 JOIN Patients p ON h.patient_amka = p.amka
17 WHERE h.discharge_date IS NOT NULL      -- μόνο ολοκληρωμένες
18 GROUP BY
19     d.name ,
20     YEAR(h.admission_date),
21     h.ken_code ,
22     p.insurance_provider
23 ORDER BY Admission_Year ASC, Department ASC, KEN_Code ASC;
```

II.α.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 629 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 γραμμές φαίνονται παρακάτω:

Department	Admission_Year	KEN_Code	Total_Base_Revenue	Total_Extra_Revenue	Insurance_Provider	Total_Hospitalizations
Δερματολογία	2023	A22Μα	1938	100	ΕΦΚΑ	1
Δερματολογία	2023	A26Μα	2467	300	ΕΦΚΑ	1
Δερματολογία	2023	B22Α	545	500	Ιδιωτική Ασφάλεια	1
Δερματολογία	2023	Δ01Χ	6452	500	ΕΦΚΑ	1
Δερματολογία	2023	Δ11Α	404	1000	Ιδιωτική Ασφάλεια	1
Δερματολογία	2023	Δ20Α	171	1800	ΕΦΚΑ	1
Δερματολογία	2023	Δ29Χ	1377	400	Ιδιωτική Ασφάλεια	1
Δερματολογία	2023	Ε11Χ	5302	0	Ιδιωτική Ασφάλεια	1
Δερματολογία	2023	H42Χ	708	700	Ανασφάλιστος	1
Δερματολογία	2023	Θ01Μ	6202	0	Ιδιωτική Ασφάλεια	1

Q1 αποτελέσματα

II.β Ερώτημα 2: Ιατροί ειδικότητας – εφημερίες και επεμβάσεις

II.β.1 Ζητούμενο

Για συγκεκριμένη ειδικότητα ιατρού, βρείτε όλους τους ιατρούς που ανήκουν σε αυτήν, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

II.β.2 Εξήγηση

Για κάθε ιατρό της επιλεγμένης ειδικότητας εμφανίζεται αν είχε εφημερία το τρέχον έτος (YEAR(CURDATE())) και ο αριθμός επεμβάσεων ως κύριος χειρουργός. LEFT JOIN ώστε να εμφανίζονται και ιατροί με 0 επεμβάσεις.

II.β.3 Υλοποίηση

```
1 SELECT
2     s.first_name,
3     s.last_name,
4     CASE
5         WHEN EXISTS (
6             SELECT 1 FROM Shift_Assignments sa
7             JOIN Shifts sh ON sa.shift_id = sh.id
8             WHERE sa.staff_amka = d.staff_amka
9                 AND YEAR(sh.shift_date) = YEAR(CURDATE())
10            ) THEN 'Ναι'
11         ELSE 'Όχι'
12     END AS Had_Shift_Current_Year,
13     COUNT(pr.id) AS Lead_Surgeries_Performed
14 FROM Doctors d
15 JOIN Staff s ON d.staff_amka = s.amka
16 LEFT JOIN Procedure_Records pr ON d.staff_amka = pr.main_surgeon_amk
17 WHERE d.specialty = 'Καρδιολογία'
18 GROUP BY
19     d.staff_amka,
20     s.first_name,
21     s.last_name;
```

II.β.4 Αποτελέσματα

Ο παραπάνω κώδικας (με παράμετρο για ειδικότητα την Καρδιολογία) επιστρέφει 19 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Φαίνονται παρακάτω:

first_name	last_name	Had_Shift_Current_Year	Lead_Surgeries_Performed
Μιχάλης	Οικονόμου	Όχι	2
Μαρία	Σταυροπούλου	Όχι	2
Μάνος	Βλάχος	Ναι	1
Νικόλας	Δημόπουλος	Ναι	1
Λουκάς	Μακρής	Ναι	1
Χάρης	Νικολάου	Όχι	1
Άρης	Νικολάου	Όχι	1
Όλγα	Παπαδοπούλου	Ναι	1
Μάρκος	Παπανδρέου	Ναι	1
Χρυσούλα	Παπαντωνίου	Όχι	1

Q2 αποτελέσματα

II.γ Ερώτημα 3: Ασθενείς με άνω των 3 νοσηλειών στο ίδιο τμήμα

II.γ.1 Ζητούμενο

Βρείτε ποιοι ασθενείς έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

II.γ.2 Εξήγηση

Το ερώτημα εντοπίζει ασθενείς με επαναλαμβανόμενες νοσηλείες (πάνω από 3) στο ίδιο τμήμα, υπολογίζοντας παράλληλα το συνολικό τους κόστος. Επειδή αναζητούμε συχνότητα ανά τμήμα και όχι συνολικά στο νοσοκομείο, η ομαδοποίηση (**GROUP BY**) γίνεται συνδυαστικά ανά ασθενή και ανά τμήμα (συμπεριλαμβάνοντας τα περιγραφικά τους πεδία για απόλυτη συμβατότητα με το strict mode της SQL).

Ο περιορισμός των αποτελεσμάτων γίνεται υποχρεωτικά με **HAVING** αντί για **WHERE**. Αυτό συμβαίνει διότι το **WHERE** φιλτράρει τις αρχικές εγγραφές πριν την ομαδοποίηση, ενώ το **HAVING** φιλτράρει τα παραγόμενα γκρουπ αφού έχει υπολογιστεί η aggregate συνάρτηση του **COUNT**.

II.γ.3 Υλοποίηση

```
1 SELECT
2     p.first_name,
3     p.last_name,
4     d.name AS Department,
5     COUNT(h.id) AS Hospitalization_Count,
6     SUM(h.total_cost) AS Total_Accumulated_Cost
7 FROM Hospitalization h
8 JOIN Patients p ON h.patient_amka = p.amka
9 JOIN Departments d ON h.department_id = d.id
10 GROUP BY
11     h.patient_amka,
12     h.department_id,
13     p.first_name,
14     p.last_name,
15     d.name
16 HAVING Hospitalization_Count > 3
17 ORDER BY Hospitalization_Count DESC;
```

II.γ.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 18 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

first_name	last_name	Department	Hospitalization_Count	Total_Accumulated_Cost
Λίτσα	Οικονόμου	Δερματολογία	5	72126
Αντώνης	Τσιπράς	Χειρουργική	5	23200
Ελένη	Αλεξίου	Δερματολογία	5	17173
Αικατερίνη	Χατζηγεωργίου	Μαιευτική	5	16430
Σωτήρης	Αλεξίου	Ουρολογία	5	15476
Θανάσης	Αντωνίου	Οφθαλμολογία	4	62759
Νατάσα	Λάμπρου	Μαιευτική	4	39349
Στέλιος	Στεφανίδης	Δερματολογία	4	21144
Ασημίνα	Νικολάου	Μαιευτική	4	12171
Στέλιος	Σταυρόπουλος	Οφθαλμολογία	4	12038

Q3 αποτελέσματα

II.δ Ερώτημα 4: Μέσος όρος αξιολογήσεων ιατρού – EXPLAIN ANALYZE

II.δ.1 Ζητούμενο

Για συγκεκριμένο ιατρό, βρείτε τον μέσο όρο αξιολογήσεων των ασθενών του (κριτήριο: Ποιότητα ιατρικής φροντίδας) και τη Συνολική εντύπωση νοσηλείας.

II.δ.2 Εκδοχή A – Κανονική εκτέλεση

II.δ.2.1 Εξήγηση

Το ερώτημα συνδέει τέσσερις πίνακες: Doctors, Staff, Evaluation_Doctor και Evaluation_Hospitalization. Το φίλτρο WHERE d.staff_amka = '...' επιτρέπει στον optimizer να ξεκινά με single-row PK lookup στο Doctors και να συνεχίζει με index lookup στο Evaluation_Doctor μέσω του fk_ed_doctor_idx.

II.δ.2.2 Υλοποίηση

```
1 EXPLAIN ANALYZE
2 SELECT
3     d.staff_amka ,
4     CONCAT(s.first_name, ' ', s.last_name) AS Doctor_Name ,
5     AVG(ed.medical_care) AS Avg_Medical_Care ,
6     AVG(eh.overall_experience) AS Avg_Overall_Experience
7 FROM Doctors d
8 JOIN Staff s ON d.staff_amka = s.amka
9 JOIN Evaluation_Doctor ed ON d.staff_amka = ed.doctor_amka
10 JOIN Evaluation_Hospitalization eh ON ed.hospitalization_id = eh.
    hospitalization_id
11 WHERE d.staff_amka = '16257864602'
12 GROUP BY d.staff_amka, s.first_name, s.last_name;
```

II.δ.2.3 Αποτελέσματα EXPLAIN ANALYZE

```
1 -> Group aggregate: avg(ed.medical_care), avg(eh.overall_experience) (cost
   =1.6 rows=1.41) (actual time=0.0245..0.0245 rows=1 loops=1)
2   -> Nested loop inner join (cost=1.4 rows=2) (actual time=0
   .0167..0.0197 rows=2 loops=1)
3     -> Index lookup on ed using fk_ed_doctor_idx (doctor_amka='
   16257864602'), with index condition: ((ed.doctor_amka = '16257864602')
   and <cache>(('16257864602' = '16257864602')) (cost=0.7 rows=2) (actual
   time=0.0117..0.0134 rows=2 loops=1)
4     -> Single-row index lookup on eh using PRIMARY (hospitalization_id=
   ed.hospitalization_id) (cost=0.3 rows=1) (actual time=0.00245..0.00245
   rows=1 loops=2)
```

II.δ.3 Εκδοχή B – Με FORCE INDEX

II.δ.3.1 Εξήγηση

Αναγκάζουμε ρητά τον optimizer να χρησιμοποιήσει το `fk_ed_doctor_idx`. Αν το plan είναι ταυτόσημο με την Εκδοχή A, αποδεικνύεται ότι ο optimizer κάνει ήδη τη βέλτιστη επιλογή.

II.δ.3.2 Υλοποίηση

```
1 EXPLAIN ANALYZE
2 SELECT
3     d.staff_amka,
4     CONCAT(s.first_name, ' ', s.last_name) AS Doctor_Name,
5     AVG(ed.medical_care) AS Avg_Medical_Care,
6     AVG(eh.overall_experience) AS Avg_Overall_Experience
7 FROM Doctors d
8 JOIN Staff s ON d.staff_amka = s.amka
9 JOIN Evaluation_Doctor ed FORCE INDEX (fk_ed_doctor_idx)
10    ON d.staff_amka = ed.doctor_amka
11 JOIN Evaluation_Hospitalization eh
12    ON ed.hospitalization_id = eh.hospitalization_id
13 WHERE d.staff_amka = '16257864602'
14 GROUP BY d.staff_amka, s.first_name, s.last_name;
```

II.δ.3.3 Αποτελέσματα EXPLAIN ANALYZE

Ταυτόσημο plan με την Εκδοχή A, με ελάχιστες διαφορές στα κόστη, χρόνους και γραμμές.

```
1 -> Group aggregate: avg(ed.medical_care), avg(eh.overall_experience) (cost
   =1.6 rows=1.41) (actual time=0.0257..0.0258 rows=1 loops=1)
2   -> Nested loop inner join (cost=1.4 rows=2) (actual time=0.0182..0.021
   rows=2 loops=1)
3     -> Index lookup on ed using fk_ed_doctor_idx (doctor_amka='
   16257864602'), with index condition: ((ed.doctor_amka = '16257864602')
   and <cache>(('16257864602' = '16257864602')))) (cost=0.7 rows=2) (actual
   time=0.0131..0.0146 rows=2 loops=1)
4     -> Single-row index lookup on eh using PRIMARY (hospitalization_id=
   ed.hospitalization_id) (cost=0.3 rows=1) (actual time=0.0025..0.0025
   rows=1 loops=2)
```

II.δ.4 Εκδοχή Γ – Με IGNORE INDEX (baseline σύγκρισης)

II.δ.4.1 Εξήγηση

Απαγορεύουμε το `fk_ed_doctor_idx` ώστε ο optimizer να αναγκαστεί σε full table scan στο `Evaluation_Doctor`. Η εκδοχή αυτή δεν είναι προτεινόμενη — χρησιμοποιείται αποκλειστικά ως *baseline* για να ποσοτικοποιηθεί η αξία του index.

II.δ.4.2 Υλοποίηση

```
1 EXPLAIN ANALYZE
2 SELECT
3     d.staff_amka,
4     CONCAT(s.first_name, ' ', s.last_name) AS Doctor_Name,
5     AVG(ed.medical_care) AS Avg_Medical_Care,
6     AVG(eh.overall_experience) AS Avg_Overall_Experience
7 FROM Doctors d
8 JOIN Staff s ON d.staff_amka = s.amka
9 JOIN Evaluation_Doctor ed IGNORE INDEX (fk_ed_doctor_idx)
10    ON d.staff_amka = ed.doctor_amka
11 JOIN Evaluation_Hospitalization eh
12    ON ed.hospitalization_id = eh.hospitalization_id
13 WHERE d.staff_amka = '16257864602'
14 GROUP BY d.staff_amka, s.first_name, s.last_name;
```

II.δ.4.3 Αποτελέσματα EXPLAIN ANALYZE

```
1 -> Group aggregate: avg(ed.medical_care), avg(eh.overall_experience) (cost
   =11.7 rows=1.83) (actual time=0.046..0.046 rows=1 loops=1)
2   -> Nested loop inner join (cost=11.4 rows=3.33) (actual time=0
   .0256..0.0404 rows=2 loops=1)
3     -> Filter: ((ed.doctor_amka = '16257864602') and <cache>((
   16257864602' = '16257864602')) (cost=10.2 rows=3.33) (actual time=0
   .0198..0.0332 rows=2 loops=1)
4       -> Table scan on ed (cost=10.2 rows=100) (actual time=0
   .0148..0.0262 rows=100 loops=1)
5         -> Single-row index lookup on eh using PRIMARY (hospitalization_id=
   ed.hospitalization_id) (cost=0.28 rows=1) (actual time=0.0029..0.00295
   rows=1 loops=2)
```

II.δ.5 Αποτελέσματα ερωτήματος

Σε κάθε περίπτωση, οι παραπάνω κώδικες (χωρίς το κομμάτι `EXPLAIN ANALYZE`) επιστρέφουν 1 γραμμή από την τρέχουσα βάση δεδομένων, οργανωμένη όπως ζητήθηκε. Φαίνεται παρακάτω:

staff_amka	last_name	Avg_Medical_Care	Avg_Overall_Experience
16257864602	Καραμανλή	5.0000	4.0000

Q4 Αποτελέσματα

II.δ.6 Σύγκριση και Ανάλυση

Μετρική	A (κανονική)	B (FORCE)	Γ (IGNORE)
Scan τύπος σε Evaluation_Doctor	Index lookup	Index lookup	Table scan
Index που χρησιμοποιείται	fk_ed_doctor_idx	fk_ed_doctor_idx	—
Εκτιμώμενο κόστος JOIN (cost=)	1.4	1.4	11.4
Γραμμές που εξετάστηκαν	2	2	100
Πραγματικός χρόνος JOIN (ms)	0.017–0.020	0.018–0.021	0.025–0.040
Συνολικός χρόνος (ms)	0.024–0.025	0.025–0.026	0.046

Table 5: Σύγκριση query plans Q4 – τιμές από EXPLAIN ANALYZE

Οι εκδοχές A και B παράγουν ταυτόσημο query plan: ο MariaDB optimizer επιλέγει αυτόματα το `fk_ed_doctor_idx`, εκτελώντας *index lookup* και εξετάζοντας μόνο 2 γραμμές από τον `Evaluation_Doctor`. Το `FORCE INDEX` δεν αλλάζει τίποτα, αποδεικνύοντας ότι ο optimizer ήδη κάνει τη βέλτιστη επιλογή.

Αξίζει να σημειωθεί ότι στο plan των εκδοχών A/B εμφανίζεται `<cache>('16257864602' = '16257864602')`: η MariaDB εφαρμόζει *constant folding* — αναγνωρίζει ότι το φίλτρο είναι σταθερή έκφραση και την αξιολογεί μία φορά, μεταφέροντάς την απευθείας στο `index condition` για μέγιστη αποδοτικότητα.

Στην Εκδοχή Γ (`IGNORE INDEX`) ο optimizer αναγκάζεται σε *full table scan* εξετάζοντας 100 γραμμές αντί 2 — αύξηση 50× στις γραμμές ανάγνωσης. Το εκτιμώμενο κόστος αυξάνεται από 1.4 σε 11.4 (αύξηση 8×), και ο πραγματικός χρόνος εκτέλεσης από 0.028ms σε 0.042ms. Σε παραγωγικό περιβάλλον με χιλιάδες αξιολογήσεις, η διαφορά αυτή κλιμακώνεται γραμμικά, καθιστώντας το `fk_ed_doctor_idx` κρίσιμο για την απόδοση του Q4.

II.ε Ερώτημα 5: Νέοι ιατροί (<35 ετών) με τις περισσότερες χειρουργικές επεμβάσεις

II.ε.1 Ζητούμενο

Βρείτε τους νέους ιατρούς (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί.

II.ε.2 Εξήγηση

Το ερώτημα εντοπίζει νέους χειρουργούς συνδυάζοντας την ηλικία τους (`s.age < 35`) και την κατηγορία της πράξης (`mpc.category = 'Χειρουργική'`), ελέγχοντας παράλληλα ότι συμμετείχαν ως βασικοί χειρουργοί (`main_surgeon_amk`). Η επιλογή `INNER JOIN` στον πίνακα `Procedure_Records` αποκλείει αυτόματα τους ιατρούς χωρίς καμία επέμβαση, χωρίς να χρειάζεται `HAVING COUNT > 0`. Το τρίτο `JOIN` στον πίνακα `Medical_Procedure_Catalog` επιτρέπει το φιλτράρισμα αποκλειστικά για χειρουργεία. Τέλος, το `GROUP BY` περιλαμβάνει τέσσερις στήλες (`AMKA`, όνομα, επώνυμο, ηλικία) ώστε να επιτευχθεί η καταμέτρηση (`COUNT`) των χειρουργείων με απόλυτη συμβατότητα στο `strict mode` της `SQL` (`ONLY_FULL_GROUP_BY`).

II.ε.3 Υλοποίηση

```
1 SELECT
2     s.first_name,
3     s.last_name,
4     s.age,
5     COUNT(pr.id) AS Surgeries_Count
6 FROM Doctors d
7 JOIN Staff s ON d.staff_amka = s.amka
8 JOIN Procedure_Records pr ON d.staff_amka = pr.main_surgeon_amk
9 JOIN Medical_Procedure_Catalog mpc ON pr.procedure_code = mpc.code
10 WHERE s.age < 35
11        AND mpc.category = 'Χειρουργική'
12 GROUP BY
13     d.staff_amka, s.first_name, s.last_name, s.age
14 ORDER BY Surgeries_Count DESC;
```

II.ε.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 40 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

first_name	last_name	age	Surgeries_Count
Γρηγόρης	Κουρής	32	5
Λουκάς	Στεφανίδης	31	4
Άρης	Αντωνίου	28	3
Νικόλας	Παπαγεωργίου	34	2
Μάνος	Δημόπουλος	34	2
Δέσποινα	Σαμαρά	28	2
Ασημίνα	Αλεξίου	34	2
Στέφανος	Κουρής	34	2
Μιχάλης	Δημητρίου	30	1
Αντώνης	Σιδέρης	34	1

Q5 αποτελέσματα

II.ζ Ερώτημα 6: Ιστορικό νοσηλειών ασθενή – EXPLAIN ANALYZE

II.ζ.1 Ζητούμενο

Για συγκεκριμένο ασθενή, βρείτε το ιστορικό νοσηλειών του, τις αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και τον μέσο όρο αξιολόγησής του.

II.ζ.2 Εκδοχή A – Κανονική εκτέλεση

II.ζ.2.1 Εξήγηση

Το ερώτημα αναζητά όλες τις νοσηλείες συγκεκριμένου ασθενή συνδέοντας τέσσερις πίνακες: Hospitalization, ICD10_Catalog (δύο φορές, για εισαγωγή και έξοδο), και Evaluation_Hospitalization. Το φίλτρο WHERE h.patient_amka = '...' επιτρέπει στον optimizer να χρησιμοποιήσει το fk_hospitalization_patient_idx για να εντοπίσει αποκλειστικά τις γραμμές του συγκεκριμένου ασθενή, αποφεύγοντας full scan του πίνακα Hospitalization.

II.ζ.2.2 Υλοποίηση

```
1 EXPLAIN ANALYZE
2 SELECT
3     h.id AS Hospitalization_ID,
4     h.admission_date,
5     h.discharge_date,
6     icd_adm.description AS Admission_Diagnosis,
7     icd_dis.description AS Discharge_Diagnosis,
8     h.total_cost,
9     (eh.nursing_care + eh.cleanliness
10      + eh.food + eh.overall_experience) / 4.0 AS Avg_Rating
11 FROM Hospitalization h
12 LEFT JOIN ICD10_Catalog icd_adm
13     ON h.admission_diagnosis_icd10 = icd_adm.code
14 LEFT JOIN ICD10_Catalog icd_dis
15     ON h.discharge_diagnosis_icd10 = icd_dis.code
16 LEFT JOIN Evaluation_Hospitalization eh
17     ON h.id = eh.hospitalization_id
18 WHERE h.patient_amka = '80589489579'
19 ORDER BY h.admission_date DESC;
```

II.ζ.2.3 Αποτελέσματα EXPLAIN ANALYZE

```
1 -> Nested loop left join (cost=12.6 rows=9) (actual time=0.0692..0.105
   rows=9 loops=1)
2   -> Nested loop left join (cost=9.45 rows=9) (actual time=0.063..0.0886
       rows=9 loops=1)
3     -> Nested loop left join (cost=6.3 rows=9) (actual time=0
        .0557..0.0703 rows=9 loops=1)
4       -> Sort: h.admission_date DESC (cost=3.15 rows=9) (actual time
          =0.048..0.0491 rows=9 loops=1)
5         -> Index lookup on h using fk_hospitalization_patient_idx (
            patient_amka='80589489579') (cost=3.15 rows=9) (actual time=0
            .0313..0.0349 rows=9 loops=1)
6           -> Filter: (h.admission_diagnosis_icd10 = icd_adm.'code') (
              cost=0.261 rows=1) (actual time=0.00201..0.00208 rows=1 loops=9)
7             -> Single-row index lookup on icd_adm using PRIMARY (code=
                h.admission_diagnosis_icd10) (cost=0.261 rows=1) (actual time=0
                .00182..0.00184 rows=1 loops=9)
8               -> Filter: (h.discharge_diagnosis_icd10 = icd_dis.'code') (cost=0
                  .261 rows=1) (actual time=0.00184..0.00191 rows=0.889 loops=9)
9                 -> Single-row index lookup on icd_dis using PRIMARY (code=
                    h.discharge_diagnosis_icd10) (cost=0.261 rows=1) (actual time=0
                    .00172..0.00176 rows=0.889 loops=9)
10                -> Single-row index lookup on eh using PRIMARY (hospitalization_id=h.id
                    ) (cost=0.261 rows=1) (actual time=0.0016..0.00162 rows=0.333 loops=9)
```


II.ζ.3 Εκδοχή B – Με FORCE INDEX

II.ζ.3.1 Εξήγηση

Αναγκάζουμε ρητά τον optimizer να χρησιμοποιήσει το `fk_hospitalization_patient_idx`. Αν το plan είναι ταυτόσημο με την Εκδοχή A, αποδεικνύεται ότι ο optimizer ήδη κάνει τη βέλτιστη επιλογή χωρίς hint.

II.ζ.3.2 Υλοποίηση

```
1 EXPLAIN ANALYZE
2 SELECT
3     h.id AS Hospitalization_ID,
4     h.admission_date,
5     h.discharge_date,
6     icd_adm.description AS Admission_Diagnosis,
7     icd_dis.description AS Discharge_Diagnosis,
8     h.total_cost,
9     (eh.nursing_care + eh.cleanliness
10      + eh.food + eh.overall_experience) / 4.0 AS Avg_Rating
11 FROM Hospitalization h FORCE INDEX (fk_hospitalization_patient_idx)
12 LEFT JOIN ICD10_Catalog icd_adm
13     ON h.admission_diagnosis_icd10 = icd_adm.code
14 LEFT JOIN ICD10_Catalog icd_dis
15     ON h.discharge_diagnosis_icd10 = icd_dis.code
16 LEFT JOIN Evaluation_Hospitalization eh
17     ON h.id = eh.hospitalization_id
18 WHERE h.patient_amka = '80589489579'
19 ORDER BY h.admission_date DESC;
```

II.ζ.3.3 Αποτελέσματα EXPLAIN ANALYZE

Ταυτόσημο plan με την Εκδοχή Α, με ελάχιστες διαφορές στα κόστη, χρόνους και γραμμές.

```
1 -> Nested loop left join (cost=12.6 rows=9) (actual time=0.0677..0.104
   rows=9 loops=1)
2   -> Nested loop left join (cost=9.45 rows=9) (actual time=0
   .0626..0.0888 rows=9 loops=1)
3     -> Nested loop left join (cost=6.3 rows=9) (actual time=0
   .0557..0.0705 rows=9 loops=1)
4       -> Sort: h.admission_date DESC (cost=3.15 rows=9) (actual time
   =0.0473..0.0483 rows=9 loops=1)
5         -> Index lookup on h using fk_hospitalization_patient_idx (
   patient_amka='80589489579 ') (cost=3.15 rows=9) (actual time=0
   .0301..0.0342 rows=9 loops=1)
6           -> Filter: (h.admission_diagnosis_icd10 = icd_adm.'code') (
   cost=0.261 rows=1) (actual time=0.00213..0.00219 rows=1 loops=9)
7             -> Single-row index lookup on icd_adm using PRIMARY (code=
   h.admission_diagnosis_icd10) (cost=0.261 rows=1) (actual time=0
   .00187..0.00189 rows=1 loops=9)
8               -> Filter: (h.discharge_diagnosis_icd10 = icd_dis.'code') (cost=0
   .261 rows=1) (actual time=0.00188..0.00193 rows=0.889 loops=9)
9                 -> Single-row index lookup on icd_dis using PRIMARY (code=
   h.discharge_diagnosis_icd10) (cost=0.261 rows=1) (actual time=0
   .00173..0.00177 rows=0.889 loops=9)
10                -> Single-row index lookup on eh using PRIMARY (hospitalization_id=h.id
   ) (cost=0.261 rows=1) (actual time=0.00157..0.00157 rows=0.333 loops=9)
```

II.ζ.4 Εκδοχή Γ – Με IGNORE INDEX (baseline σύγκρισης)

II.ζ.4.1 Εξήγηση

Απαγορεύουμε το `fk_hospitalization_patient_idx` ώστε ο optimizer να αναγκαστεί σε full table scan στον `Hospitalization`. Η εκδοχή αυτή χρησιμοποιείται αποκλειστικά ως *baseline* για να ποσοτικοποιηθεί η αξία του index — δεν είναι προτεινόμενη.

II.ζ.4.2 Υλοποίηση

```
1 EXPLAIN ANALYZE
2 SELECT
3     h.id AS Hospitalization_ID,
4     h.admission_date,
5     h.discharge_date,
6     icd_adm.description AS Admission_Diagnosis,
7     icd_dis.description AS Discharge_Diagnosis,
8     h.total_cost,
9     (eh.nursing_care + eh.cleanliness
10      + eh.food + eh.overall_experience) / 4.0 AS Avg_Rating
11 FROM Hospitalization h IGNORE INDEX (fk_hospitalization_patient_idx)
12 LEFT JOIN ICD10_Catalog icd_adm
13     ON h.admission_diagnosis_icd10 = icd_adm.code
14 LEFT JOIN ICD10_Catalog icd_dis
15     ON h.discharge_diagnosis_icd10 = icd_dis.code
16 LEFT JOIN Evaluation_Hospitalization eh
17     ON h.id = eh.hospitalization_id
18 WHERE h.patient_amka = '80589489579'
19 ORDER BY h.admission_date DESC;
```

II.ζ.4.3 Αποτελέσματα EXPLAIN ANALYZE

```
1 -> Nested loop left join (cost=255 rows=660) (actual time=0.245..0.31 rows
2   =9 loops=1)
3   -> Nested loop left join (cost=173 rows=660) (actual time=0.239..0.292
4     rows=9 loops=1)
5     -> Nested loop left join (cost=90.4 rows=660) (actual time=0
6       .236..0.251 rows=9 loops=1)
7       -> Sort: h.admission_date DESC (cost=7.85 rows=660) (actual
8         time=0.223..0.224 rows=9 loops=1)
9         -> Filter: (h.patient_amka = '80589489579 ') (cost=7.85
10          rows=660) (actual time=0.0404..0.21 rows=9 loops=1)
11          -> Table scan on h (cost=7.85 rows=660) (actual time=0
              .0366..0.179 rows=660 loops=1)
              -> Filter: (h.admission_diagnosis_icd10 = icd_adm.'code') (
                  cost=0.252 rows=1) (actual time=0.00264..0.00268 rows=1 loops=9)
                  -> Single-row index lookup on icd_adm using PRIMARY (code=
                      h.admission_diagnosis_icd10) (cost=0.252 rows=1) (actual time=0
                          .00246..0.00248 rows=1 loops=9)
                          -> Filter: (h.discharge_diagnosis_icd10 = icd_dis.'code') (cost=0
                              .252 rows=1) (actual time=0.00448..0.00451 rows=0.889 loops=9)
                              -> Single-row index lookup on icd_dis using PRIMARY (code=
                                  h.discharge_diagnosis_icd10) (cost=0.252 rows=1) (actual time=0
                                      .00431..0.00432 rows=0.889 loops=9)
                                      -> Single-row index lookup on eh using PRIMARY (hospitalization_id=h.id
                                          ) (cost=0.252 rows=1) (actual time=0.00177..0.00178 rows=0.333 loops=9)
```

II.ζ.5 Αποτελέσματα ερωτήματος

Σε κάθε περίπτωση, οι παραπάνω κώδικες (χωρίς το κομμάτι EXPLAIN ANALYZE) επιστρέφουν 9 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 5 φαίνονται παρακάτω:

Hospitalization_ID	admission_date	discharge_date	Admission_Diagnosis	Discharge_Diagnosis	total_cost	Avg_Rating
605	2026-01-29T12:34:00.000Z	2026-02-07T12:34:00.000Z	Κάταγμα του μηριαίου οστού	Καρδιακή ανεπάρκεια	2638	2.2500
488	2026-01-10T06:19:00.000Z		Εγκεφαλικό έμφρακτο		887	
4	2025-10-14T15:40:00.000Z	2025-10-23T15:40:00.000Z	Κακόηθες νεόπλασμα βρόγχου και πνεύμονα	Άλλες διαταραχές των υγρών, των ηλεκτρολυτών και της οξεοβασικής ισορροπίας	4296	
67	2025-08-31T00:18:00.000Z	2025-09-16T00:18:00.000Z	Διάρροια και γαστρεντερίτιδα που θεωρείται λοιμώδους προέλευσης	Εκφυλιστική αρθρίτιδα του ισχίου	2026	
585	2024-09-04T13:40:00.000Z	2024-09-14T13:40:00.000Z	Φροντίδα της μητέρας για γνωστή ανωμαλία ή υποψία ανωμαλίας των πυελικών οργάνων	Πολλαπλή σκλήρυνση	1157	3.2500

Q6 αποτελέσματα

II.ζ.6 Σύγκριση και Ανάλυση

Μετρική	A (κανονική)	B (FORCE)	Γ (IGNORE)
Scan type σε Hospitalization	Index lookup	Index lookup	Table scan
Index που χρησιμοποιείται	fk_hosp_patient_idx	fk_hosp_patient_idx	—
Εκτιμώμενο κόστος (cost=)	3.15	3.15	255
Γραμμές που εξετάστηκαν	9	9	660
Πραγματικός χρόνος scan (ms)	0.031–0.035	0.030–0.034	0.037–0.210
Συνολικός χρόνος (ms)	0.069–0.105	0.068–0.104	0.245–0.310

Σύγκριση query plans Q6 – τιμές από EXPLAIN ANALYZE

Οι εκδοχές A και B παράγουν ταυτόσημο query plan: ο MariaDB optimizer επιλέγει αυτόματα το `fk_hospitalization_patient_idx`, εκτελώντας *index lookup* και εξετάζοντας μόνο τις 9 νοσηλείες του συγκεκριμένου ασθενή από σύνολο 660. Το `FORCE INDEX` δεν αλλάζει τίποτα, αποδεικνύοντας ότι ο optimizer ήδη κάνει τη βέλτιστη επιλογή.

Η Εκδοχή Γ (`IGNORE INDEX`) αποκαλύπτει την αξία του ευρετηρίου: χωρίς αυτό ο optimizer αναγκάζεται σε *full table scan* εξετάζοντας και τις 660 εγγραφές του `Hospitalization`. Το εκτιμώμενο κόστος εκτοξεύεται από 3.15 σε 255 (αύξηση 81×), ενώ ο πραγματικός χρόνος scan αυξάνεται από 0.035 ms σε 0.210 ms (αύξηση 6×). Σε παραγωγικό περιβάλλον με δεκάδες χιλιάδες νοσηλείες, η διαφορά κλιμακώνεται γραμμικά, καθιστώντας το `fk_hospitalization_patient_idx` κρίσιμο για την απόδοση του Q6.

II.η Ερώτημα 7: Δραστικές ουσίες – αλλεργίες ασθενών και φάρμακα

II.η.1 Ζητούμενο

Βρείτε για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικών ασθενών.

II.η.2 Εξήγηση

Το ερώτημα ξεκινά από τον πίνακα `Active_Substances` και προσαρτά δύο ανεξάρτητους πίνακες με `LEFT JOIN`: τις αλλεργίες ασθενών και τα φάρμακα που περιέχουν την ουσία. Η χρήση `LEFT JOIN` (αντί `INNER`) εξασφαλίζει ότι εμφανίζονται και ουσίες χωρίς αλλεργίες ή χωρίς φάρμακα. Το `COUNT(DISTINCT ...)` σε κάθε στήλη είναι απαραίτητο: ο συνδυασμός των δύο `LEFT JOINs` δημιουργεί Cartesian product για κάθε ουσία (αν μία ουσία έχει 3 αλλεργικούς και 4 φάρμακα, το `JOIN` παράγει 12 γραμμές), οπότε χωρίς `DISTINCT` τα counts θα ήταν λανθασμένα.

II.η.3 Υλοποίηση

```
1 SELECT
2     a.name                                AS Active_Substance ,
3     COUNT(DISTINCT pa.patient_amka)      AS Allergic_Patients ,
4     COUNT(DISTINCT mhs.medicine_code)    AS Medicines_Containing
5 FROM Active_Substances a
6 LEFT JOIN Patient_Allergies pa ON a.id = pa.substance_id
7 LEFT JOIN Medicine_has_Substances mhs ON a.id = mhs.substance_id
8 GROUP BY a.id, a.name
9 ORDER BY Allergic_Patients DESC, Medicines_Containing DESC;
```

II.η.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 40 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

Active_Substance	Allergic_Patients_Count	Medicines_Containing_Count
Silybum Marianum D15	2	1
Rhamnus Frangula E Cortice Ferm 33e Ø (Hab	2	1
Acidum Succinicum D30	2	1
Vs. 33b)	2	2
Amlodipine Besilate	1	9
Hypericum Perforatum L.	1	1
Tincture From Agnus Castus Fruit (1:10)	1	2
Celecoxib	1	1
Acidum Cis-Aconiticum D30	1	1
Chlorobutanol	1	1

Q7 αποτελέσματα

II.θ Ερώτημα 8: Προσωπικό χωρίς εφημερία σε συγκεκριμένη ημερομηνία και τμήμα

II.θ.1 Ζητούμενο

Βρείτε το προσωπικό (ιατροί, νοσηλευτές, διοικητικό προσωπικό) που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

II.θ.2 Εξήγηση

Η δομή του ερωτήματος αντικατοπτρίζει την ετερογένεια του σχήματος: οι γιατροί ανήκουν σε τμήματα μέσω πίνακα σχέσης (M:N), ενώ νοσηλευτές και διοικητικοί έχουν άμεσο FK. Γι' αυτό χρειάζονται τρία ανεξάρτητα EXISTS με OR αντί ενός JOIN. Το EXISTS (αντί JOIN) εξασφαλίζει ότι κάθε μέλος προσωπικού εμφανίζεται μία φορά, ακόμα κι αν ανήκει σε πολλαπλά τμήματα. Το NOT EXISTS επαληθεύει απουσία βάρδιας: ελέγχει αν υπάρχει εγγραφή στο Shift_Assignments για τη συγκεκριμένη ημερομηνία και τμήμα ταυτόχρονα. Η υποερώτηση SELECT id FROM Departments WHERE name = ... μεταφράζει το όνομα του τμήματος σε αριθμητικό id δυναμικά, χωρίς hardcoded τιμές.

II.θ.3 Υλοποίηση

```
1 SELECT
2     s.amka,
3     s.first_name,
4     s.last_name,
5     s.staff_type
6 FROM Staff s
7 WHERE (
8     EXISTS (
9         SELECT 1 FROM Doctor_has_Department dhd
10        WHERE dhd.doctor_amka = s.amka
11              AND dhd.department_id = (SELECT id FROM Departments WHERE name
12 = 'Καρδιολογία' LIMIT 1)
13     )
14    OR
15    EXISTS (
16        SELECT 1 FROM Nurses n
17        WHERE n.staff_amka = s.amka
18              AND n.department_id = (SELECT id FROM Departments WHERE name =
19 'Καρδιολογία' LIMIT 1)
20    )
21    OR
22    EXISTS (
23        SELECT 1 FROM Admin_Staff adm
24        WHERE adm.staff_amka = s.amka
25              AND adm.department_id = (SELECT id FROM Departments WHERE name
26 = 'Καρδιολογία' LIMIT 1)
27    )
28 )
29 AND NOT EXISTS (
30     SELECT 1
31     FROM Shift_Assignments sa
32     JOIN Shifts sh ON sa.shift_id = sh.id
33     WHERE sa.staff_amka = s.amka
```

```

31      AND sh.shift_date = '2026-05-10'
32      AND sa.department_id = (SELECT id FROM Departments WHERE name = 'Κ
αρδιολογία' LIMIT 1)
33 );

```

II.θ.4 Αποτελέσματα

Ο παραπάνω κώδικας (με παραμέτρους ημερομηνία 2026-05-10 και τμήμα Καρδιολογίας) επιστρέφει 59 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

amka	first_name	last_name	staff_type
21765210406	Στέφανος	Αλεξίου	Admin
85477032871	Στέλλα	Αντωνίου	Admin
51735224837	Ασημίνα	Γεωργίου	Admin
42918702058	Μάριος	Κουρής	Admin
97699532547	Δέσποινα	Πατέρα	Admin
59105678414	Βασιλική	Πολίτη	Admin
56113394612	Παρασκευή	Πολίτη	Admin
67051873160	Φωτεινή	Τσιπρά	Admin
05690882454	Αγγελική	Αλεξίου	Doctor
86151265928	Νικολέτα	Αλεξίου	Doctor

Q8 αποτελέσματα

II.ι Ερώτημα 9: Ασθενείς με ίδιο αριθμό νοσηλευτικών ημερών (>15) εντός έτους

II.ι.1 Ζητούμενο

Βρείτε ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών.

II.ι.2 Εξήγηση

Το ερώτημα χρησιμοποιεί δύο CTEs σε αλυσίδα. Το πρώτο, *PatientYearlyStays*, υπολογίζει το σύνολο νοσηλευτικών ημερών ανά ασθενή ανά έτος με `SUM(DATEDIFF(...))`, φιλτράροντας μόνο ολοκληρωμένες νοσηλείες (`discharge_date IS NOT NULL`) και αποκλείοντας σύνολα ≤ 15 ημερών μέσω `HAVING`. Το δεύτερο CTE, *RankedStays*, εισάγει *window function*: το `COUNT(*) OVER(PARTITION BY Hosp_Year, Total_Days)` μετράει πόσοι ασθενείς έχουν ακριβώς τον ίδιο αριθμό ημερών την ίδια χρονιά, χωρίς να σπάει τις γραμμές όπως θα έκανε ένα `GROUP BY`. Το τελικό `WHERE Patients_With_Same_Days > 1` κρατά μόνο ασθενείς που έχουν τουλάχιστον ένα «ταίρι» — δηλαδή κάποιον άλλο με ίδιο σύνολο ημερών το ίδιο έτος.

II.ι.3 Υλοποίηση

```
1 WITH PatientYearlyStays AS (  
2     SELECT  
3         patient_amka,  
4         YEAR(admission_date) AS Hosp_Year,  
5         SUM(DATEDIFF(discharge_date, admission_date)) AS Total_Days  
6     FROM Hospitalization  
7     WHERE discharge_date IS NOT NULL  
8     GROUP BY patient_amka, YEAR(admission_date)  
9     HAVING Total_Days > 15  
10 ),  
11 RankedStays AS (  
12     SELECT  
13         patient_amka,  
14         Hosp_Year,  
15         Total_Days,  
16         COUNT(*) OVER(PARTITION BY Hosp_Year, Total_Days) AS  
17         Patients_With_Same_Days  
18     FROM PatientYearlyStays  
19 )  
20 SELECT  
21     r.patient_amka,  
22     p.first_name,  
23     p.last_name,  
24     r.Hosp_Year,  
25     r.Total_Days  
26 FROM RankedStays r  
27 JOIN Patients p ON r.patient_amka = p.amka  
28 WHERE r.Patients_With_Same_Days > 1  
29 ORDER BY r.Hosp_Year DESC, r.Total_Days DESC;
```

II.1.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 135 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

patient_amka	first_name	last_name	Hosp_Year	Total_Days
83469164857	Σοφία	Αλεξίου	2026	26
36375574917	Παύλος	Δημόπουλος	2026	26
41108869443	Αλεξάνδρα	Αναγνώστου	2026	20
20623582337	Χριστίνα	Μακρή	2026	20
96478600439	Καλλιόπη	Πετροπούλου	2026	20
31171054912	Ασημίνα	Νικολάου	2025	38
51566751641	Σωτήρης	Μανωλάς	2025	38
96470658295	Ιωάννα	Δημητρίου	2025	37
17897974522	Σοφία	Μανωλά	2025	37
23773124028	Αναστασία	Σταυροπούλου	2025	33

Q9 αποτελέσματα

II.κ Ερώτημα 10: Top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα

II.κ.1 Ζητούμενο

Πολλοί ασθενείς λαμβάνουν συνδυασμούς φαρμάκων. Βρείτε τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης.

II.κ.2 Εξήγηση

Το ερώτημα λύνει ένα μη-τετριμμένο πρόβλημα: την εύρεση συνδυασμών στοιχείων (ζευγών) αποφεύγοντας τις διπλοεγγραφές. Το πρώτο CTE, `HospPrescriptions`, εξάγει το σύνολο δραστικών ουσιών ανά νοσηλεία — ένα φάρμακο μπορεί να περιέχει πολλές ουσίες, γι' αυτό το JOIN στο `Medicine_has_Substances` συνδυάζεται με `DISTINCT` για να αποφευχθούν τα duplicates. Στο δεύτερο CTE, `SubstancePairs`, γίνεται self-join του `HospPrescriptions` κατά `hosp_id`: για κάθε νοσηλεία βρίσκονται όλα τα ζεύγη ουσιών. Η συνθήκη `hp1.substance_id < hp2.substance_id` εξασφαλίζει ότι το ζεύγος (A, B) μετράται ακριβώς μία φορά — αποτρέπει τόσο τα duplicates (A,B) + (B,A) όσο και τα self-pairs (A,A). Το τελικό `GROUP BY ... ORDER BY ... LIMIT 3` δίνει τα τρία πιο συχνά ζεύγη.

II.κ.3 Υλοποίηση

```
1 WITH HospPrescriptions AS (  
2     SELECT DISTINCT h.id AS hosp_id, mhs.substance_id  
3     FROM Hospitalization h  
4     JOIN Prescriptions p  
5         ON h.patient_amka = p.patient_amka  
6         AND p.start_date >= DATE(h.admission_date)  
7         AND (h.discharge_date IS NULL  
8             OR p.start_date <= DATE(h.discharge_date))  
9     JOIN Medicine_has_Substances mhs ON p.medicine_code = mhs.  
10    medicine_code  
11 ),  
12 SubstancePairs AS (  
13     SELECT hp1.substance_id AS sub1, hp2.substance_id AS sub2  
14     FROM HospPrescriptions hp1  
15     JOIN HospPrescriptions hp2  
16         ON hp1.hosp_id = hp2.hosp_id  
17         AND hp1.substance_id < hp2.substance_id  
18 )  
19 SELECT as1.name AS Substance_1, as2.name AS Substance_2, COUNT(*) AS  
20     Frequency  
21 FROM SubstancePairs sp  
22 JOIN Active_Substances as1 ON sp.sub1 = as1.id  
23 JOIN Active_Substances as2 ON sp.sub2 = as2.id  
24 GROUP BY sp.sub1, sp.sub2, as1.name, as2.name  
25 ORDER BY Frequency DESC  
26 LIMIT 3;
```

II.κ.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 3 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Φαίνονται παρακάτω:

Substance_1	Substance_2	Frequency
Abacavir	Lamivudine	9
Adapalene	Benzoyl Peroxide	8
Colecalciferol	Alendronic Acid	6

Q10 αποτελέσματα

II.λ Ερώτημα 11: Ιατροί με τουλάχιστον 5 λιγότερες επεμβάσεις (τρέχον έτος)

II.λ.1 Ζητούμενο

Βρείτε όλους τους ιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος.

II.λ.2 Εξήγηση

Το ερώτημα αντιμετωπίζει ένα κλασικό πρόβλημα SQL: χρήση aggregate τιμής (global maximum) μέσα σε WHERE clause. Αυτό δεν επιτρέπεται άμεσα, γι' αυτό χρησιμοποιείται το τέχνασμα CROSS JOIN: το CTE MaxSurgeries επιστρέφει μία γραμμή με την μέγιστη τιμή, και το CROSS JOIN την «κολλά» σε κάθε γραμμή του αποτελέσματος ώστε να είναι διαθέσιμη στο WHERE. Το LEFT JOIN στο DoctorSurgeries (αντί για INNER) εξασφαλίζει ότι ιατροί χωρίς επεμβάσεις (0 εγγραφές) εμφανίζονται με NULL, το οποίο το COALESCE(..., 0) μετατρέπει σε 0. Έτσι και ιατρός με 0 επεμβάσεις εμφανίζεται στα αποτελέσματα, αρκεί να απέχει ≥ 5 από τον πρώτο. Το YEAR(CURDATE()) εξασφαλίζει δυναμική αναφορά στο τρέχον έτος χωρίς hardcoded τιμή.

II.λ.3 Υλοποίηση

```
1 WITH DoctorSurgeries AS (  
2     SELECT main_surgeon_amk AS doctor_amka, COUNT(id) AS total_surgeries  
3     FROM Procedure_Records  
4     WHERE YEAR(start_time) = YEAR(CURDATE())  
5     GROUP BY main_surgeon_amk  
6 ),  
7 MaxSurgeries AS (SELECT MAX(total_surgeries) AS max_surg FROM  
8     DoctorSurgeries)  
9 SELECT s.first_name, s.last_name,  
10     COALESCE(ds.total_surgeries, 0) AS Surgeries_Performed  
11 FROM Doctors d  
12 JOIN Staff s ON d.staff_amka = s.amka  
13 LEFT JOIN DoctorSurgeries ds ON d.staff_amka = ds.doctor_amka  
14 CROSS JOIN MaxSurgeries ms  
15 WHERE COALESCE(ds.total_surgeries, 0) <= (ms.max_surg - 5)  
16 ORDER BY Surgeries_Performed DESC;
```

II.λ.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 303 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

first_name	last_name	Surgeries_Performed
Στέφανος	Κουρής	2
Μιχάλης	Δημητρίου	2
Μάριος	Δημόπουλος	1
Όλγα	Παπαδοπούλου	1
Μαρία	Μαυρίδου	1
Σοφία	Μανωλά	1
Αλέξανδρος	Παπακωνσταντίνου	1
Αναστασία	Μανωλά	1
Νικολέτα	Βλάχου	1
Χριστίνα	Πετροπούλου	1

Q11 αποτελέσματα

II.μ Ερώτημα 12: Απαιτούμενο προσωπικό ανά τμήμα, βάρδια, υποκλάση (εβδομάδα)

II.μ.1 Ζητούμενο

Βρείτε τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και ανά βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκλάση προσωπικού (ιατροί ανά ειδικότητα, νοσηλευτές ανά βαθμίδα, διοικητικό προσωπικό ανά ρόλο).

II.μ.2 Εξήγηση

Η πολυπλοκότητα του ερωτήματος πηγάζει από τη στρατηγική ετερογένεια: ιατροί, νοσηλευτές και διοικητικοί έχουν διαφορετικές «υποκλάσεις» (*specialty*, *rank*, *role*) αποθηκευμένες σε διαφορετικούς πίνακες. Τα τρία παράλληλα **LEFT JOIN** λύνουν αυτό το πρόβλημα: κάθε μέλος προσωπικού ανήκει σε ακριβώς έναν από αυτούς τους πίνακες, οπότε τα άλλα δύο **JOIN** επιστρέφουν **NULL**. Το **CASE st.staff_type** επιλέγει ποια στήλη να δείξει με βάση τον τύπο — εναλλακτικά θα μπορούσε να χρησιμοποιηθεί **COALESCE**, αλλά αυτό θα δημιουργούσε πρόβλημα αν ένας τύπος είχε **NULL** τιμή (π.χ. στην ειδικότητα) για άλλο λόγο. Η ίδια **CASE** έκφραση επαναλαμβάνεται στο **GROUP BY** γιατί σε αυστηρή λειτουργία (*strict mode*) δεν επιτρέπεται η αναφορά σε *alias* του **SELECT**. Το **WEEK(...)** αντί του **BETWEEN** είναι συνειδητή επιλογή: επιστρέφει όλες τις ημέρες της εβδομάδας χωρίς να χρειάζεται υπολογισμός των ακραίων ημερομηνιών.

II.μ.3 Υλοποίηση

```
1 SELECT
2     d.name AS Department, sh.shift_date, sh.shift_type,
3     st.staff_type AS Main_Profession,
4     CASE st.staff_type
5         WHEN 'Doctor' THEN doc.specialty
6         WHEN 'Nurse'   THEN n.'rank'
7         WHEN 'Admin'   THEN adm.role
8         ELSE 'Άγνωστο'
9     END AS Subclass_Role,
10    COUNT(sa.staff_amka) AS Staff_Count
11 FROM Shifts sh
12 JOIN Shift_Assignments sa ON sh.id = sa.shift_id
13 JOIN Departments d      ON sa.department_id = d.id
14 JOIN Staff st           ON sa.staff_amka = st.amka
15 LEFT JOIN Doctors doc   ON st.amka = doc.staff_amka
16 LEFT JOIN Nurses n      ON st.amka = n.staff_amka
17 LEFT JOIN Admin_Staff adm ON st.amka = adm.staff_amka
18 WHERE WEEK(sh.shift_date) = WEEK('2026-05-10')
19     AND YEAR(sh.shift_date) = 2026
20 GROUP BY
21     d.name, sh.shift_date, sh.shift_type, st.staff_type,
22     CASE st.staff_type WHEN 'Doctor' THEN doc.specialty
23                       WHEN 'Nurse'   THEN n.'rank'
24                       WHEN 'Admin'   THEN adm.role ELSE 'Άγνωστο'
25     END
26 ORDER BY sh.shift_date, sh.shift_type, d.name, Main_Profession;
```

II.μ.4 Αποτελέσματα

Ο παραπάνω κώδικας (με παράμετρο ημερομηνία 2026-05-10) επιστρέφει 1657 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

Department	shift_date	shift_type	Main_Profession	Subclass_Role	Staff_Count
Δερματολογία	2026-05-09T21:00:00.000Z	Afternoon	Admin	Λογιστής	2
Δερματολογία	2026-05-09T21:00:00.000Z	Afternoon	Doctor	Δερματολογία	1
Δερματολογία	2026-05-09T21:00:00.000Z	Afternoon	Doctor	Παθολογία	2
Δερματολογία	2026-05-09T21:00:00.000Z	Afternoon	Nurse	Προϊστάμενος	1
Δερματολογία	2026-05-09T21:00:00.000Z	Afternoon	Nurse	Νοσηλεύτης	5
Επείγοντα	2026-05-09T21:00:00.000Z	Afternoon	Admin	Γραμματέας	2
Επείγοντα	2026-05-09T21:00:00.000Z	Afternoon	Doctor	Καρδιολογία	2
Επείγοντα	2026-05-09T21:00:00.000Z	Afternoon	Doctor	Παθολογία	1
Επείγοντα	2026-05-09T21:00:00.000Z	Afternoon	Nurse	Προϊστάμενος	2
Επείγοντα	2026-05-09T21:00:00.000Z	Afternoon	Nurse	Βοηθός Νοσηλεύτη	2

Q12 αποτελέσματα

II.ν Ερώτημα 13: Ιεραρχία εποπτείας ιατρού (Recursive CTE)

II.ν.1 Ζητούμενο

Βρείτε για κάθε ιατρό όλη την ιεραρχία εποπτείας του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα.

II.ν.2 Εξήγηση

Το **Recursive CTE** είναι ο μοναδικός τρόπος στην SQL για να διατρέξει κανείς ένα δέντρο αυθαίρετου βάθους χωρίς procedural κώδικα. Αποτελείται από δύο τμήματα συνδεδεμένα με **UNION ALL**:

- **Anchor** (βάση της αναδρομής): επιλέγει όλους τους ιατρούς που έχουν επόπτη (`supervisor_amka IS NOT NULL`), αρχικοποιώντας το επίπεδο ιεραρχίας σε 1.
- **Recursive step**: κάνει JOIN του αποτελέσματος της προηγούμενης επανάληψης με τον πίνακα `Doctors` για να ανεβεί ένα επίπεδο στην αλυσίδα εποπτείας. Η συνθήκη `WHERE d.supervisor_amka IS NOT NULL` σταματά την αναδρομή όταν φτάσουμε σε Διευθυντή (ο οποίος δεν έχει επόπτη).

Το **UNION ALL** (και όχι το **UNION DISTINCT**) είναι πρακτικά **υποχρεωτικό** στα recursive CTEs της MariaDB για λόγους απόδοσης και αυστηρότητας. Το τελικό αποτέλεσμα: κάθε ιατρός εμφανίζεται τόσες φορές όσα τα επίπεδα εποπτείας πάνω από αυτόν (π.χ. ιατρός με 3 επίπεδα ιεραρχίας από πάνω του εμφανίζεται 3 φορές, μία για κάθε επόπτη).

II.ν.3 Υλοποίηση

```
1 WITH RECURSIVE SupervisionHierarchy AS (  
2     SELECT staff_amka AS doctor_amka, supervisor_amka, 1 AS  
3     supervision_level  
4     FROM Doctors WHERE supervisor_amka IS NOT NULL  
5  
6     UNION ALL  
7  
8     SELECT sh.doctor_amka, d.supervisor_amka, sh.supervision_level + 1  
9     FROM SupervisionHierarchy sh  
10    JOIN Doctors d ON sh.supervisor_amka = d.staff_amka  
11    WHERE d.supervisor_amka IS NOT NULL  
12 )  
13 SELECT s1.last_name AS Doctor, s2.last_name AS Supervisor,  
14        sh.supervision_level AS Level  
15 FROM SupervisionHierarchy sh  
16 JOIN Staff s1 ON sh.doctor_amka = s1.amka  
17 JOIN Staff s2 ON sh.supervisor_amka = s2.amka  
18 ORDER BY s1.last_name, sh.supervision_level;
```

II.ν.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 566 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

Doctor_LastName	Supervisor_LastName	Hierarchy_Level
Αλεξίου	Παπαντωνίου	1
Αλεξίου	Δημητρίου	1
Αλεξίου	Βασιλείου	1
Αλεξίου	Νικολάου	1
Αλεξίου	Νικολάου	1
Αλεξίου	Χατζηγεωργίου	1
Αλεξίου	Αναγνώστου	1
Αλεξίου	Σιδέρης	1
Αλεξίου	Νικολάου	2
Αλεξίου	Δημητρίου	2

Q13 αποτελέσματα

II.ξ Ερώτημα 14: ICD-10 διαγνώσεις με ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη

II.ξ.1 Ζητούμενο

Βρείτε ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος.

II.ξ.2 Εξήγηση

Η λογική βασίζεται σε *self-join* *χρονοσειράς*: το CTE `YearlyAdmissions` υπολογίζει τον αριθμό εισαγωγών ανά (ICD-10 κωδικός, έτος), κρατώντας όσα έχουν ≥ 5 περιστατικά μέσω του `HAVING`. Ο *self-join* που ακολουθεί ζευγαρώνει κάθε έτος με το επόμενο (`y2.hosp_year = y1.hosp_year + 1`) για τον ίδιο κωδικό, και το `WHERE y1.total_cases = y2.total_cases` κρατά μόνο τα ζεύγη με ίδιο πλήθος. Η γεννήτρια δεδομένων χρησιμοποιεί την `HOSP_ICD_POOL` (25 κοινοί κωδικοί) για να εξασφαλίσει επαρκή επαναληψιμότητα ώστε το ερώτημα να επιστρέφει αποτελέσματα — με 11.000 διαφορετικούς κωδικούς στο σύνολο του ICD-10, κανείς δεν θα επαναλαμβανόταν εύκολα.

II.ξ.3 Υλοποίηση

```
1 WITH YearlyAdmissions AS (  
2     SELECT  
3         admission_diagnosis_icd10 AS icd_code,  
4         YEAR(admission_date)      AS hosp_year,  
5         COUNT(*)                  AS total_cases  
6     FROM Hospitalization  
7     WHERE admission_diagnosis_icd10 IS NOT NULL  
8     GROUP BY admission_diagnosis_icd10, YEAR(admission_date)  
9     HAVING total_cases >= 5  
10 )  
11 SELECT  
12     y1.icd_code AS ICD10_Code, c.description AS Disease_Description,  
13     y1.hosp_year AS Year_1, y2.hosp_year AS Year_2,  
14     y1.total_cases AS Cases_Per_Year  
15 FROM YearlyAdmissions y1  
16 JOIN YearlyAdmissions y2  
17     ON y1.icd_code = y2.icd_code  
18     AND y2.hosp_year = y1.hosp_year + 1  
19 JOIN ICD10_Catalog c ON y1.icd_code = c.code  
20 WHERE y1.total_cases = y2.total_cases  
21 ORDER BY y1.icd_code, y1.hosp_year;
```

II.ξ.4 Αποτελέσματα

Ο παραπάνω κώδικας επιστρέφει 6 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Φαίνονται παρακάτω:

ICD10_Code	Disease_Description	Year_1	Year_2	Cases_Per_Year
E11	Μη ινσουλινοεξαρτώμενος σακχαρώδης διαβήτης	2024	2025	11
O34	Φροντίδα της μητέρας για γνωστή ανωμαλία ή υποψία ανωμαλίας των πυελικών οργάνων	2025	2026	5
C50	Κακόηθες νεόπλασμα του μαστού	2023	2024	8
K92	Άλλα νοσήματα του πεπτικού συστήματος	2023	2024	11
C18	Κακόηθες νεόπλασμα του παχέος εντέρου [κόλου]	2023	2024	5
S06	Ενδοκρανιακός τραυματισμός	2023	2024	5

Q14 Αποτελέσματα

II.ο Ερώτημα 15: Κατανομή triage ανά επίπεδο επείγοντος

II.ο.1 Ζητούμενο

Βρείτε την κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα.

II.ο.2 Εξήγηση

Το ερώτημα αντιμετωπίζει δύο σχεδιαστικές προκλήσεις.

- Πρόβλημα 1 — Αντιστοίχιση triage με νοσηλεία: Δεν υπάρχει άμεσο FK μεταξύ `Triage_Records` και `Hospitalization`. Η αντιστοίχιση γίνεται μέσω `LEFT JOIN` κατά `patient_amka` με παράθυρο 24 ωρών. Χρησιμοποιείται `MIN(h.id)` καθώς και `MIN(h.admission_date)` για να εξασφαλιστεί μία γραμμή ανά triage, ακόμα κι αν ο ασθενής έχει πολλές νοσηλείες. Χωρίς το `MIN`, κάθε triage θα πολλαπλασιαζόταν για κάθε νοσηλεία του ασθενή.
- Πρόβλημα 2 — `GROUP_CONCAT` με implicit grouping: Αν το `LEFT JOIN` με τον πίνακα `Departments` γινόταν άμεσα στο τελικό `SELECT`, η παρουσία του `d.name` θα ανάγκαζε το `GROUP BY` να συμπεριλάβει και το τμήμα, παράγοντας πολλές γραμμές ανά `urgency_level`. Η λύση είναι το ενδιάμεσο CTE `DeptNames` που προ-joinάει τα ονόματα τμημάτων πριν το τελικό aggregation: έτσι το `GROUP_CONCAT(DISTINCT dept_name)` λειτουργεί σωστά εντός του `GROUP BY urgency_level`. Το `SUM(hosp_id IS NOT NULL)` εκμεταλλεύεται το γεγονός ότι στη MariaDB τα boolean expressions επιστρέφουν 0/1, αντικαθιστώντας το πιο verbose `SUM(CASE WHEN hosp_id IS NOT NULL THEN 1 ELSE 0 END)`.

II.ο.3 Μέρος Α'

II.ο.3.1 Υλοποίηση

```
1  -- ΜΕΡΟΣ Α: Σύνολο ανά urgency level (5 γραμμές)
2  WITH TriageMatched AS (
3      SELECT
4          t.id AS triage_id, t.urgency_level, t.arrival_time,
5          MIN(h.id) AS hosp_id, MIN(h.admission_date) AS admission_date,
6          MIN(h.department_id) AS dept_id
7      FROM Triage_Records t
8      LEFT JOIN Hospitalization h
9          ON t.patient_amka = h.patient_amka
10         AND h.admission_date >= t.arrival_time
11         AND h.admission_date < DATE_ADD(t.arrival_time, INTERVAL 24
12         HOUR)
13     GROUP BY t.id, t.urgency_level, t.arrival_time
14 ),
15 DeptNames AS (
16     SELECT tm.urgency_level, tm.triage_id, tm.hosp_id, tm.arrival_time,
17         tm.admission_date, d.name AS dept_name
18     FROM TriageMatched tm
19     LEFT JOIN Departments d ON tm.dept_id = d.id
20 )
21 SELECT
22     urgency_level AS Urgency_Level,
23     COUNT(triage_id) AS Total_Triage,
24     SUM(hosp_id IS NOT NULL) AS Admitted_Count,
25     ROUND(SUM(hosp_id IS NOT NULL) / COUNT(triage_id) * 100, 1)
26         AS Hospitalization_Rate_Pct,
27     ROUND(AVG(
28         CASE WHEN hosp_id IS NOT NULL
29             THEN TIMESTAMPDIFF(MINUTE, arrival_time, admission_date)
30             END), 0) AS Avg_Wait_Min,
31     GROUP_CONCAT(DISTINCT dept_name ORDER BY dept_name SEPARATOR ' | ')
32         AS Referred_Departments
33 FROM DeptNames
34 GROUP BY urgency_level
35 ORDER BY urgency_level;
```

Π.ο.3.2 Αποτελέσματα

Ο κώδικας του πρώτου μέρους επιστρέφει 5 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Φαίνονται παρακάτω:

Urgency_Level	Total_Triage	Admitted_Count	Hospitalization_Rate_Pct	Avg_Wait_Min	Referred_Departments
1	167	163	97.6	407	Δερματολογία Επείγοντα Καρδιολογία Μαιευτική ΜΕΘ Νευρολογία Νεφρολογία Ογκολογία Ορθοπεδική Ουρολογία Οφθαλμολογία Παιδιατρική Πνευμονολογία Χειρουργική ΩΡΛ
2	156	156	100.0	408	Δερματολογία Επείγοντα Καρδιολογία Μαιευτική ΜΕΘ Νευρολογία Νεφρολογία Ογκολογία Ορθοπεδική Ουρολογία Οφθαλμολογία Παιδιατρική Πνευμονολογία Χειρουργική ΩΡΛ
3	142	110	77.5	416	Δερματολογία Καρδιολογία Μαιευτική Νευρολογία Νεφρολογία Ογκολογία Ορθοπεδική Ουρολογία Οφθαλμολογία Παιδιατρική Πνευμονολογία Χειρουργική ΩΡΛ
4	149	105	70.5	394	Δερματολογία Καρδιολογία Μαιευτική Νευρολογία Νεφρολογία Ογκολογία Ορθοπεδική Ουρολογία Οφθαλμολογία Παιδιατρική Πνευμονολογία Χειρουργική ΩΡΛ
5	178	126	70.8	415	Δερματολογία Καρδιολογία Μαιευτική Νευρολογία Νεφρολογία Ογκολογία Ορθοπεδική Ουρολογία Οφθαλμολογία Παιδιατρική Πνευμονολογία Χειρουργική ΩΡΛ

Q15 Αποτελέσματα πρώτου μέρους

II.ο.4 Μέρος Β'

II.ο.4.1 Υλοποίηση

```
1  -- ΜΕΡΟΣ Β: Αναλυτικά ανά urgency + τμήμα
2  WITH TriageMatched AS (
3      SELECT t.id, t.urgency_level, t.arrival_time,
4             MIN(h.admission_date) AS admission_date,
5             MIN(h.department_id) AS dept_id
6      FROM Triage_Records t
7      LEFT JOIN Hospitalization h
8           ON t.patient_amka = h.patient_amka
9           AND h.admission_date >= t.arrival_time
10          AND h.admission_date < DATE_ADD(t.arrival_time, INTERVAL 24
11          HOUR)
12      GROUP BY t.id, t.urgency_level, t.arrival_time
13  )
14  SELECT
15      tm.urgency_level AS Urgency_Level,
16      COALESCE(d.name, '- Αποχώρησε -') AS Referred_Department,
17      COUNT(*) AS Cases,
18      ROUND(AVG(CASE WHEN tm.dept_id IS NOT NULL
19                  THEN TIMESTAMPDIFF(MINUTE, tm.arrival_time, tm.
20                  admission_date)
21                  END), 0) AS Avg_Wait_Min
22  FROM TriageMatched tm
23  LEFT JOIN Departments d ON tm.dept_id = d.id
24  GROUP BY tm.urgency_level, d.name
25  ORDER BY tm.urgency_level, Cases DESC;
```

II.ο.4.2 Αποτελέσματα

Ο κώδικας του δεύτερου μέρους επιστρέφει 73 γραμμές από την τρέχουσα βάση δεδομένων, οργανωμένες και ταξινομημένες όπως ζητήθηκε. Οι πρώτες 10 φαίνονται παρακάτω:

Urgency_Level	Referred_Department	Cases	Avg_Wait_Min
1	ΜΕΘ	25	411
1	Επείγοντα	19	355
1	Δερματολογία	17	416
1	Ουρολογία	14	381
1	ΩΡΛ	13	403
1	Πνευμονολογία	12	399
1	Οφθαλμολογία	10	515
1	Νεφρολογία	9	469
1	Νευρολογία	8	401
1	Χειρουργική	8	442

Q15 Αποτελέσματα δεύτερου μέρους

III. Πηγές, Παραπομπές και Χρήση Εργαλείων Τεχνητής Νοημοσύνης

III.α Επίσημες πηγές δεδομένων αναφοράς

Τα δεδομένα αναφοράς της βάσης εισήχθησαν αυτούσια από τις παρακάτω επίσημες πηγές, όπως ορίζει η εκφώνηση:

- ICD-10: Στατιστική Ταξινόμηση Νόσων και Σχετικών Προβλημάτων Υγείας, Υπουργείο Υγείας
- KEN: Κλειστά Ενοποιημένα Νοσήλια, Υπουργείο Υγείας
- Ιατρικές Πράξεις: Ελληνική Ονοματολογία και Κωδικοποίηση Ιατρικών Πράξεων, Υπουργείο Υγείας
- EMA Article 57: Βάση δεδομένων εγκεκριμένων φαρμάκων Ευρωπαϊκού Οικονομικού Χώρου, European Medicines Agency

III.β Τεκμηρίωση και κοινότητες

Κατά την υλοποίηση χρησιμοποιήθηκαν οι παρακάτω πηγές για τεκμηρίωση, επίλυση αποριών και καλές πρακτικές:

- Σελίδα μαθήματος ΒΔ 2025-2026 (ΕΜΠ):
Εκφώνηση εργασίας, οδηγίες υποβολής και ανακοινώσεις.
- MariaDB / MySQL Documentation:
Τεκμηρίωση για triggers, stored procedures, recursive CTEs, window functions και EXPLAIN ANALYZE.
- Stack Overflow:
Επίλυση συγκεκριμένων προβλημάτων SQL και Node.js (named placeholders, LOAD DATA, bulk INSERT σύνταξη).
- Database Administrators Stack Exchange (dba.stackexchange):
Ερωτήματα σχετικά με σχεδιασμό triggers, index strategy και query optimization.
- TeX Stack Exchange (tex.stackexchange):
Επίλυση προβλημάτων L^AT_EX για τη σύνταξη της αναφοράς (`\lstlisting`, Greek text, tabular formatting).

III.γ Χρήση Εργαλείων Τεχνητής Νοημοσύνης

Σύμφωνα με τις οδηγίες της εκφώνησης, δηλώνεται ρητά η χρήση του εργαλείου Claude (Anthropic, μοντέλο Claude Sonnet 4.6) και Gemini (Google, μοντέλο Gemini Pro 3.1) κατά την εκπόνηση της παρούσας εργασίας.

III.γ.1 Τρόπος συμβολής

- Εντοπισμός σφαλμάτων: Διάγνωση και διόρθωση σφαλμάτων σε SQL triggers (apostrophe escaping, labeled BEGIN blocks), Node.js endpoints (named placeholders, LIMIT interpolation) και λογικών σφαλμάτων στα ερωτήματα (Q15 GROUP BY implicit grouping).
- Βελτιστοποίηση: Μετατροπή 233.584 single-row INSERT statements σε bulk INSERTs (batch=100), εισαγωγή @load_mode για bypass triggers κατά το load, και βελτιστοποίηση run_all.bat.
- Υλοποίηση: Συγγραφή τμημάτων του backend (autofill endpoint, EXPLAIN ANALYZE endpoint, doctors-by-department) και frontend (admit modal, shift validation, EXPLAIN button).
- Αναφορά: Σύνταξη εξηγήσεων ερωτημάτων, πινάκων σύγκρισης EXPLAIN ANALYZE και τμημάτων τεκμηρίωσης triggers/indexes.

III.γ.2 Επίβλεψη και επικύρωση

Όλος ο παραγόμενος κώδικας και το περιεχόμενο ελέγχθηκε, κατανοήθηκε και επικυρώθηκε από την ομάδα. Οι σχεδιαστικές αποφάσεις (schema, triggers, αρχιτεκτονική 3-layer enforcement, επιλογή indexes) αναλύθηκαν και συζητήθηκαν διαδραστικά, με την ομάδα να λαμβάνει τις τελικές αποφάσεις.